

【数据结构应用题】打卡表参考文档

使用说明：强化阶段（二轮复习）打卡表，并没有所谓的“标准答案”，希望大家尽量独立完成打卡任务，实在没思路再参考这个文档。在第二轮复习中，应注重“输出”训练，切忌不要形成惰性思维，遇到问题就翻答案。忘记的知识先翻王道书复习，再尝试独立完成打卡表任务，尽量靠自己解决。大家加油！

注1：强化阶段（二轮复习）打卡表，将陆续更新复习任务。找不到这个资料的同学可以咨询王道班主任天天老师。

强化阶段（二轮复习）“应用题”备考打卡表

模块	考点	优先级	索引	训练任务	需要翻书?	是否已完成	备注
真题训练	应用题	必做	2009应用题	做408真题_2009_41题, 即王道书 6.4.6_大题_8	☐	☐	先做历年真题, 感受应用题的考法, 这会让你备考更有方向性
			2010应用题	做408真题_2010_41题, 即王道书 7.5.5_大题_6	☐	☐	
			2011应用题	做408真题_2011_41题, 即王道书 6.4.6_大题_9	☐	☐	
			2012应用题	做408真题_2012_41题, 即王道书 5.5.3_大题_2	☐	☐	
			2013应用题	做408真题_2013_41题, 即王道书 7.2.4_大题_7	☐	☐	
			2015应用题	做408真题_2015_42题, 即王道书 6.2.6_大题_5	☐	☐	
			2016应用题	做408真题_2016_42题, 即王道书 5.4.4_大题_7	☐	☐	
			2017应用题	做408真题_2017_42题, 即王道书 6.4.6_大题_11	☐	☐	
			2018应用题	做408真题_2018_42题, 即王道书 6.4.6_大题_12	☐	☐	
			2019应用题	做408真题_2019_42题, 即王道书 3.2.5_大题_4	☐	☐	
			2014应用题	做408真题_2014_42题, 即王道书 6.4.6_大题_10	☐	☐	复习完计网, 再来做这个题
		必做	2020应用题	做408真题_2020_42题, 即王道书 5.5.3_大题_3	☐	☐	可以留下最近三年的真题, 二轮复习结束后用于自模考。 当然, 也可以直接做, 没必要“舍不得”做真题
			2021应用题	做408真题_2021_42题, 即王道书 8.6.3_大题_5	☐	☐	
			2022应用题	做408真题_2022_42题, 即王道书 8.4.3_大题6 (注: 23版王道书未收录这道题)	☐	☐	
					☐	☐	
“数组”的应用	对称矩阵的压缩存储	必做	1.1.1	自己动手创造, 画一个5行5列的 对称矩阵	☐	☐	尚未在应用题中考过“对称矩阵压缩存储” 未来应用题有可能将无向图的邻接矩阵、对称矩阵压缩存储一起考察
			1.1.2	按“行优先”压缩存储上述矩阵, 画出一维数组的样子	☐	☐	
			1.1.3	写出元素 i, j 与 数组下标之间的对应关系	☐	☐	
			1.1.4	按“列优先”压缩存储上述矩阵, 画出一维数组的样子	☐	☐	
			1.1.5	写出元素 i, j 与 数组下标之间的对应关系	☐	☐	
			1.1.6	假设你的对称矩阵表示一个 无向图 , 画出无向图的样子	☐	☐	
	上/下三角矩阵的压缩存储	高优先级	1.2.1	自己动手创造, 画一个5行5列的 下三角矩阵	☐	☐	2011年41题曾考过“三角矩阵的压缩存储”
			1.2.2	按“行优先”压缩存储上述矩阵, 画出一维数组的样子	☐	☐	
			1.2.3	写出元素 i, j 与 数组下标之间的对应关系	☐	☐	
			1.2.4	按“列优先”压缩存储上述矩阵, 画出一维数组的样子	☐	☐	
			1.2.5	写出元素 i, j 与 数组下标之间的对应关系	☐	☐	
			1.2.6	假设你的对称矩阵表示一个 有向图 , 画出有向图的样子	☐	☐	
	三对角矩阵的压缩存储	低优先级	1.3.1	自己动手创造, 画一个5行5列的 三对角矩阵	☐	☐	“三对角矩阵”在应用题中的可考察性较弱, 很难和其他考点一起综合考, 更可能考小题
			1.3.2	按“行优先”压缩存储上述矩阵, 画出一维数组的样子	☐	☐	
			1.3.3	写出元素 i, j 与 数组下标之间的对应关系	☐	☐	
			1.3.4	按“列优先”压缩存储上述矩阵, 画出一维数组的样子	☐	☐	
			1.3.5	写出元素 i, j 与 数组下标之间的对应关系	☐	☐	

注2：该文档具有目录索引，可以快速定位跳转，方便你使用

目录

DS强化任务打卡表参考文档

Ch1 数组的应用

1.1 对称矩阵的压缩存储

1.1.1 自己动手创造, 画一个5行5列的...

1.1.2 + 1.1.4 按“行优先/列优先”压缩...

1.1.3 + 1.1.5 写出元素 i, j 与 数组下...

1.1.6 假设你的对称矩阵表示一个无向...

1.2 上/下三角矩阵的压缩存储

1.2.1 自己动手创造, 画一个5行5列的...

1.2.2 + 1.2.4 按“行优先/列优先”压缩...

1.2.3 + 1.2.5 写出元素 i, j 与 数组下...

Ch1 数组的应用

在应用题中，“数组”常结合“矩阵压缩存储”考察，此类题目需要注意以下条件：

- ◇ 行优先存储 or 列优先存储？
- ◇ 矩阵下标从 1 or 0 开始？——若题目未特别说明，矩阵下标默认从 1 开始
- ◇ 数组下标从 0 or 1 开始？——若题目未特别说明，数组下标默认从 0 开始

1.1 对称矩阵的压缩存储

若考察“对称矩阵的压缩存储”，除了关注行优先 or 列优先，还需注意题目要求压缩存储的是下三角区域 or 上三角区域：

1.1.1 自己动手创造，画一个 5 行 5 列的对称矩阵

	1	2	3	4	5
1	0	5	3	3	4
2	5	0	2	6	7
3	3	2	0	8	9
4	3	6	8	0	6
5	4	7	9	6	0

5行5列的对称矩阵， $a_{i,j} = a_{j,i}$

1.1.2 + 1.1.4 按“行优先/列优先”压缩存储上述矩阵，画出一维数组的样子



1.1.3 + 1.1.5 写出元素 i, j 与 数组下标之间的对应关系

设 n 行 n 列矩阵下标从 1 开始，数组下标从 0 开始。矩阵元素 i, j 和数组元素 k 的对应关系为：

① 压缩存储下三角区域+主对角线，行优先存储

$$k = \begin{cases} \frac{i(i-1)}{2} + j - 1, & i \geq j \text{ (下三角区和主对角线元素)} \\ \frac{j(j-1)}{2} + i - 1, & i < j \text{ (上三角区)} \end{cases}$$

② 压缩存储下三角区域+主对角线，列优先存储

$$k = \begin{cases} \frac{(j-1)(2n-j+2)}{2} + (i-j), & i \geq j \text{ (下三角区和主对角线元素)} \\ \frac{(i-1)(2n-i+2)}{2} + (j-i), & i < j \text{ (上三角区)} \end{cases}$$

③ 压缩存储上三角区域+主对角线，行优先存储

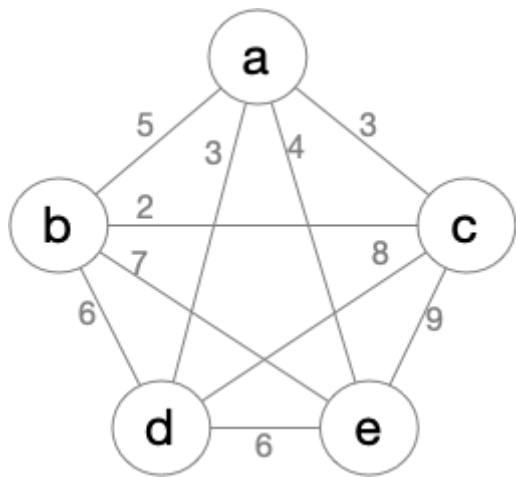
$$k = \begin{cases} \frac{(i-1)(2n-i+2)}{2} + (j-i), & i \leq j \text{ (上三角区和主对角线元素)} \\ \frac{(j-1)(2n-j+2)}{2} + (i-j), & i > j \text{ (下三角区)} \end{cases}$$

④ 压缩存储上三角区域+主对角线，列优先存储

$$k = \begin{cases} \frac{j(j-1)}{2} + i - 1, & i \leq j \text{ (上三角区和主对角线元素)} \\ \frac{i(i-1)}{2} + j - 1, & i > j \text{ (下三角区)} \end{cases}$$

注：基于对称矩阵的特性可知，只需将式①的 i 、 j 互换即可得到④；只需将式②的 i 、 j 互换即可得到③；

1.1.6 假设你的对称矩阵表示一个无向图，画出无向图的样子



注：“无向图的邻接矩阵”刚好是一个“对称矩阵”，因此这两个考点很可能会结合考察

1.2 上/下三角矩阵的压缩存储

1.2.1 自己动手创造，画一个 5 行 5 列的下三角矩阵

	1	2	3	4	5
1	1	0	0	0	0
2	5	1	0	0	0
3	3	2	1	0	0
4	3	6	8	1	0
5	4	7	9	6	1

1.2.2 + 1.2.4 按“行优先/列优先”压缩存储上述矩阵，画出一维数组的样子

行优先压缩存储：

1	5	1	3	2	1	3	6	8	1	4	7	9	6	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

 ①

数组下标：

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

列优先压缩存储：

1	5	3	3	4	1	2	6	7	1	8	9	1	6	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

 ②

数组下标：

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

1.2.3 + 1.2.5 写出元素 i, j 与 数组下标之间的对应关系

设 n 行 n 列矩阵下标从 1 开始，数组下标从 0 开始。矩阵元素 i, j 和数组元素 k 的对应关系为：

① 压缩存储下三角区域+主对角线，行优先存储

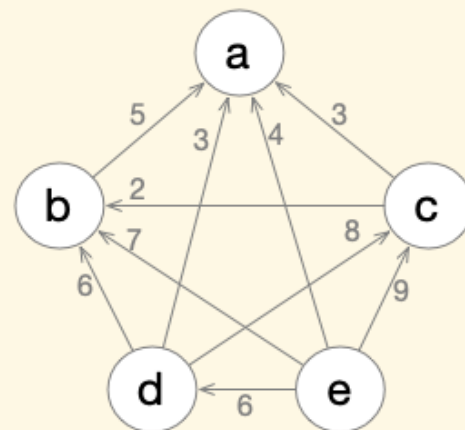
$$k = \begin{cases} \frac{i(i-1)}{2} + j - 1, & i \geq j \text{ (下三角区和主对角线元素)} \\ \frac{n(n+1)}{2}, & i < j \text{ (上三角区)} \end{cases}$$

② 压缩存储下三角区域+主对角线，列优先存储

$$k = \begin{cases} \frac{(j-1)(2n-j+2)}{2} + (i - j), & i \geq j \text{ (下三角区和主对角线元素)} \\ \frac{n(n+1)}{2}, & i < j \text{ (上三角区)} \end{cases}$$

注：若一个有向图的邻接矩阵仅有下三角区域非空，主对角线和上三角区都为空（如下所示），则该有向图一定是有向无环图。因为所有的“有向边”都是大编号顶点指向小编号顶点，所以对于任意两个顶点 i 、 j （假设 $i > j$ ），则有可能存在 i 到 j 的路径，但一定不存在 j 到 i 的路径，因此不可能形成“环”。

	1	2	3	4	5
1	0	0	0	0	0
2	5	0	0	0	0
3	3	2	0	0	0
4	3	6	8	0	0
5	4	7	9	6	0



1.3 三对角矩阵的压缩存储

注：任务 1.3.1~1.3.5 请参考王道书（或考点精讲课件）

“三对角矩阵（带状矩阵）”更有可能考选择题，不太可能在应用题考察。从出题人的视角猜测，三对角矩阵不太方便和其他考点综合考察。

相比之下，“对称矩阵”可能和“无向图”综合考察；“三角矩阵”可能和“有向无环图”综合考察

Ch2 栈、队列的应用

2.1 栈的定义和基本操作实现

2.1.1 + 2.1.2 定义顺序栈，并实现基本操作

顺序栈的定义和实现，参考王道书 3.1.2

```

//定义栈结点

typedef struct SNode{           //定义单链表结点类型
    int data;                   //每个节点存放一个数据元素
    struct SNode *next;        //指针指向下一个节点
}SNode, *LiStack;

//初始化一个链栈（单链表实现，栈顶在链头）
bool InitStack(LiStack &S) {
    S = (SNode *) malloc(sizeof(SNode)); //分配一个头结点
    S->next = NULL;                       //头结点之后暂时还没有节点
    return true;
}

//判断栈是否为空
bool StackEmpty(LiStack S){
    if(S->next==NULL)    //头结点后面没有结点
        return true;    //返回 true，表示栈为空
    else
        return false;
}

//入栈（本质上是单链表的“头插法”）
bool Push (LiStack &S, int x){
    SNode * p = (SNode *) malloc(sizeof(SNode)); //新分配一个结
    点
    p->data = x;           //存入新元素
    p->next = S->next;
    S->next = p;           //新结点插入到头结点后面
    return true;
}

//出栈（本质上是单链表的“头删法”）
bool Pop (LiStack &S, int &x){
    if (StackEmpty(S))    //栈空，出栈操作失败
        return false;

    SNode * p = S->next;   //栈不空，链头结点出栈
    x = p->data;            //x 返回栈顶元素
    S->next = p->next;      //头删法，栈顶元素“断链”
    free(p);               //释放结点
}

```


2.1.5 + 2.1.6 定义链栈，并实现基本操作（要求双链表实现，栈顶在链尾）

王道计算机考研

```

//定义栈结点

typedef struct DbSNode{                //定义双链表结点类型
    int data;                          //每个节点存放一个数据元素
    struct DbSNode *last,*next;       //指向前后两个结点
}DbSNode;

typedef struct DbLiStack{              //双链表实现的栈（栈顶在链尾）
    struct DbSNode *head, *rear;      //两个指针，分别指向链头、链尾
}DbLiStack, *DbStack;

//初始化一个链栈（单链表实现，栈顶在链头）
bool InitDbStack(DbStack &S) {
    S = (DbLiStack *) malloc(sizeof(DbLiStack)); //初始化一个链
    栈（双链表实现，栈顶在链尾）

    DbSNode * p = (DbSNode *) malloc(sizeof(DbSNode)); //新建一个头
    结点

    p->next = NULL;                //头结点之后暂时还没有节点
    p->last = NULL;                //头结点之前没有节点

    S->head = p;
    S->rear = p;

    return true;
}

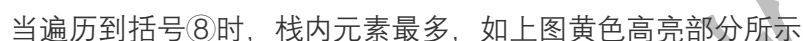
//判断栈是否为空
bool DbStackEmpty(DbStack S){
    if(S->head == S->rear) //头指针和尾指针指向同一个结点
        return true;     //返回 true，表示栈为空
    else
        return false;
}

//入栈（在双链表链尾插入）
bool DbPush (DbStack &S, int x){
    DbSNode * p = (DbSNode *) malloc(sizeof(DbSNode)); //新分
    配一个结点

    p->data = x;                //存入新元素
    p->next = NULL;

```

① a + ② b ÷ ③ (c + d) ④ ⑤) × ⑥ [⑦ (e + f) + g] ⑧ × ⑨ (h + i) ⑩] ÷ j ⑪ ⑫ ⑬ ⑭



注：你会发现，你可以理解很多算法的执行过程，也可以画图模拟，但是如果题目要求你用文字描述算法执行过程，会很难表述。因此平时要注意练习，不能只是“看懂别人写的”，更重要的是“自己能写出来”。

初始时栈为空。

①当遍历到左括号时，将其压入栈顶。

②当遍历到右括号时，将栈顶元素出栈，并判断出栈的左括号与当前遍历的右括号是否匹配（左小括号应与右小括号匹配、左中括号应与右中括号匹配）：如果不匹配，则算法结束，括号匹配失败；如果匹配，则继续遍历下一个括号元素。

当遍历完算术表达式中所有括号元素后，若栈为空，则括号匹配成功；若栈非空，则括号匹配失败。

2.2.1 + 2.2.2 写代码：定义顺序存储的队列，并实现基本操作

参考王道书 3.2.2

2.2.3 + 2.2.4 定义链式存储的队列，并实现基本操作

参考王道书 3.2.3

Ch3 树的应用

3.1 二叉树的性质、定义、画图

3.1.1 总结二叉树的度、树高、结点数等属性之间的关系

通过王道书 5.2.3 课后小题来复习 “二叉树的性质”

3.1.2 定义顺序存储的二叉树（从数组下标 1 开始存储）

二叉树顺序存储的数据结构定义，需要注意以下两点：

1. 二叉树的结点用数组存储，每个结点需要标记 “是否为空”
2. 各结点的数组下标隐含结点关系，要能根据一个结点的数组下标 i ，计算出其父节点、左孩子、右孩子的数组下标

```

typedef struct TreeNode {
    int data;          //结点中的数据元素
    bool isEmpty;      //结点是否为空
} TreeNode;

//初始化顺序存储的二叉树，所有结点标记为"空"
void InitSqBiTree (TreeNode t[], int length) {
    for (int i=0; i<length; i++){
        t[i].isEmpty=true;
    }
}

int main() {
    TreeNode t[100];      //定义一棵顺序存储的二叉树
    InitSqBiTree(t, 100); //初始化为空树
    //...
}

```

3.1.3~3.1.5 实现函数，找到结点 i 的父结点、左孩子、右孩子

```

//判断下标为 index 的结点是否为空
bool isEmpty(TreeNode t[], int length, int index){
    if (index >= length || index < 1) return true;    //下标超出合法
范围
    return t[index].isEmpty;
}

//找到下标为 index 的结点的左孩子，并返回左孩子的下标，如果没有左孩子，则返
回 -1
int getLchild(TreeNode t[], int length, int index){
    int lChild = index * 2;    //如果左孩子存在，则左孩子的下标一定是
index * 2
    if (isEmpty(t, length, lChild)) return -1;    //左孩子为空
    return lChild;
}

//找到下标为 index 的结点的右孩子，并返回右孩子的下标，如果没有右孩子，则返
回 -1
int getRchild(TreeNode t[], int length, int index){
    int rChild = index * 2 + 1;    //如果右孩子存在，则
右孩子的下标一定是 index * 2 + 1
    if (isEmpty(t, length, rChild)) return -1;    //右孩子为空
    return rChild;
}

//找到下标为 index 的结点的父节点，并返回父节点的下标，如果没有父节点，则返
回 -1
int getFather(TreeNode t[], int length, int index){
    if (index == 1) return -1;    //根节点没有父节点
    int father = index / 2;    //如果父节点存在，则父节点的下
标一定是 index/2，整数除法会自动向下取整
    if (isEmpty(t, length, father)) return -1;
    return father;
}

```

3.1.6 利用上述三个函数，实现先/中/后序遍历

对于链式存储的二叉树，先序、中序、后序遍历的递归遍历代码实现很简单，大家都会
对于顺序存储的二叉树，利用上面 封装的函数，也可以实现递归遍历

王道计算机考研

```

//从下标为 index 的结点开始先序遍历
void PreOrderSqTree (TreeNode *t, int length, int index){
    if (isEmpty(t, length, index))    //当前为空节点
        return;

    visitNode(t[index]);    //访问结点
    PreOrderSqTree(t, length, getLchild(t, length, index));
//先序遍历左子树
    PreOrderSqTree(t, length, getRchild(t, length, index));
//先序遍历右子树
}

//从下标为 index 的结点开始中序遍历
void InOrderSqTree (TreeNode *t, int length, int index){
    if (isEmpty(t, length, index))    //当前为空节点
        return;

    InOrderSqTree(t, length, getLchild(t, length, index));
//中序遍历左子树
    visitNode(t[index]);    //访问结点
    InOrderSqTree(t, length, getRchild(t, length, index));
//中序遍历右子树
}

//从下标为 index 的结点开始后序遍历
void PostOrderSqTree (TreeNode *t, int length, int index){
    if (isEmpty(t, length, index))    //当前为空节点
        return;

    PostOrderSqTree(t, length, getLchild(t, length, index));
//后序遍历左子树
    PostOrderSqTree(t, length, getRchild(t, length, index));
//后序遍历右子树
    visitNode(t[index]);    //访问结点
}

int main(){
    TreeNode t[100];    //定义一棵顺序存储的二叉树
    InitSqBiTree(t, 100);    //初始化为空树
    //...在空二叉树中插入数据
    //...略

    InOrderSqTree (t, 100, 1);    //从根节点 1 出发, 进行中序遍历
}

```


3.1.7 定义顺序存储的二叉树（从数组下标 0 开始存储）

与 3.1.2 完全相同

3.1.8~3.1.10 实现函数，找到结点 i 的父结点、左孩子、右孩子（结点从下标 0 开始存储）

与 3.1.3~3.1.5 思路相同，只不过结点编号的计算有些区别而已

若顺序二叉树从数组下标 0 开始存储结点，则：

- 结点 i 的父结点编号为 $\lfloor (i+1)/2 \rfloor - 1$
- 结点 i 的左孩子编号为 $\lfloor (i+1)*2 \rfloor - 1 = 2*i + 1$
- 结点 i 的右孩子编号为 $\lfloor (i+1)*2+1 \rfloor - 1 = 2*i + 2$

```

//判断下标为 index 的结点是否为空
bool isEmpty(TreeNode t[], int length, int index){
    if (index >= length || index < 0) return true;    //下标超出合法
范围
    return t[index].isEmpty;
}

//找到下标为 index 的结点的左孩子, 并返回左孩子的下标, 如果没有左孩子, 则返
回 -1
int getLchild(TreeNode t[], int length, int index){
    int lChild = index * 2 + 1;    //如果左孩子存在, 则左孩子的下标一定
是 index * 2
    if (isEmpty(t, length, lChild)) return -1;    //左孩子为空
    return lChild;
}

//找到下标为 index 的结点的右孩子, 并返回右孩子的下标, 如果没有右孩子, 则返
回 -1
int getRchild(TreeNode t[], int length, int index){
    int rChild = index * 2 + 2;    //如果右孩子存在, 则
右孩子的下标一定是 index * 2 + 1
    if (isEmpty(t, length, rChild)) return -1;    //右孩子为空
    return rChild;
}

//找到下标为 index 的结点的父节点, 并返回父节点的下标, 如果没有父节点, 则返
回 -1
int getFather(TreeNode t[], int length, int index){
    if (index == 1) return -1;    //根节点没有父节点
    int father = (index + 1) / 2 - 1;    //如果父节点存在,
则父节点的下标一定是 index/2, 整数除法会自动向下取整
    if (isEmpty(t, length, father)) return -1;
    return father;
}

```

3.1.11 利用上述三个函数，实现先/中/后序遍历

与 3.1.6 一模一样，不再赘述

3.2 树的性质

3.2.1 总结树的度、树高、结点数等属性之间的关系

通过王道书 5.1.4、5.4.4 课后小题来复习 “树和森林的性质”

3.3 树的定义、画图

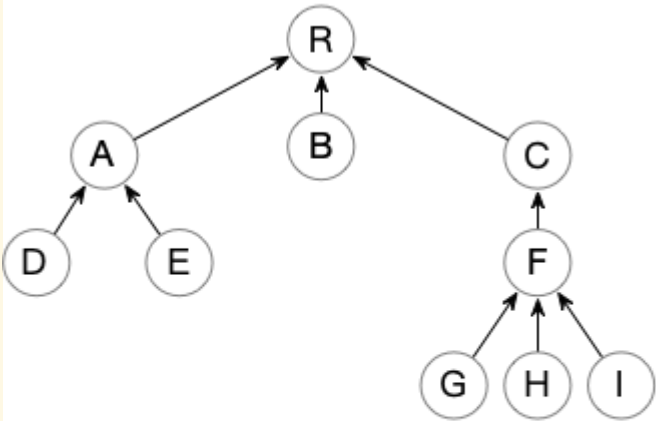
3.3.1 使用“双亲表示法”定义顺序存储的树（以及森林）

参考王道书 5.4.1 双亲表示法的代码描述

双亲表示法的存储结构描述如下：

```
#define MAX_TREE_SIZE 100           //树中最多结点数
typedef struct{                      //树的结点定义
    ElemType data;                  //数据元素
    int parent;                      //双亲位置域
}PTNode;
typedef struct{                      //树的类型定义
    PTNode nodes[MAX_TREE_SIZE];    //双亲表示
    int n;                          //结点数
}PTree;
```

双亲表示法可以表示 “多叉树” ， 如下所示：



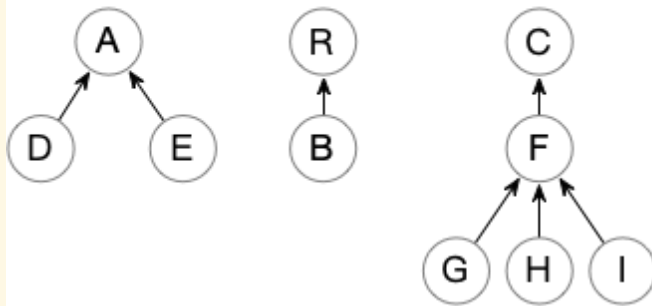
由10个结点组成的树

	data	parent
0	R	-1
1	A	0
2	B	0
3	C	0
4	D	1
5	E	1
6	F	3
7	G	6
8	H	6
9	I	6

双亲表示法表示“树”

显然，双亲表示法也可以表示“森林”

只需令森林中的每棵树根结点的 parent 值为 -1 即可，如下所示：



由10个结点组成的森林，共计3棵树

	data	parent
0	R	-1
1	A	-1
2	B	0
3	C	-1
4	D	1
5	E	1
6	F	3
7	G	6
8	H	6
9	I	6

双亲表示法表示“森林”

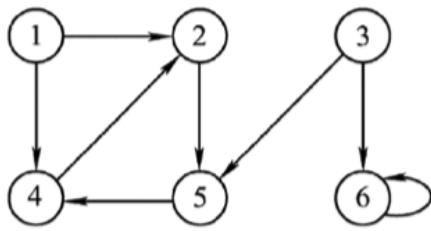
注意上面这个例子，如果未来考“森林”的应用题，很可能考双亲表示法的数据结构定义+画图

3.3.2 使用“孩子表示法”，定义链式存储的树（以及森林）

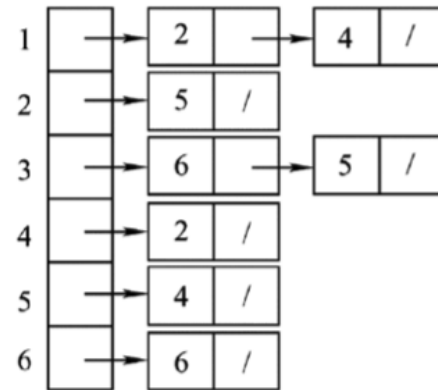
```
//Child 表示下一个孩子的信息
typedef struct Child{
    int index;           //孩子编号
    struct Child * next; //下一个孩子
} Child;

//TreeNode 用于保存结点信息
typedef struct TreeNode {
    char data;           //结点信息
    Child * firstChild;  //指向第一个孩子
} TreeNode;

TreeNode tree[10]; //定义一棵拥有 10 个结点的树（孩子表示法）
```



◆ (a)有向图 G



(b)有向图 G 的邻接表的表示

◆ 图 6.8 有向图邻接表表示法实例

如果一个图拥有 10 个顶点，则可以这么定义其数据结构（请与本文档 3.3.2 “孩子表示法”的数据结构定义对比）：

```

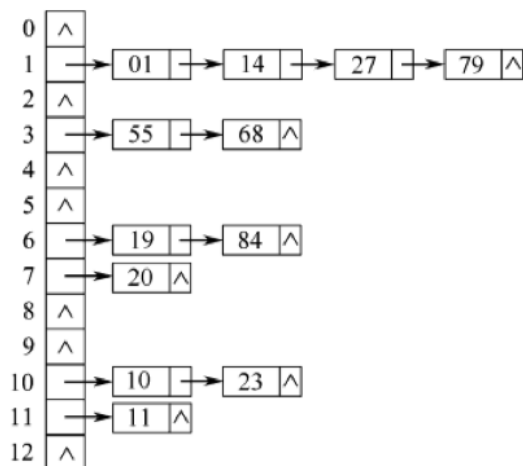
typedef struct ArcNode{           //边表结点
    int adjvex;                   //该弧所指向的顶点的位置
    struct ArcNode *next;        //指向下一条弧的指针
    //InfoType info;             //网的边权值
}ArcNode;

typedef struct VNode{            //顶点表结点
    char data;                   //顶点信息
    ArcNode *first;              //指向第一条依附该顶点的弧的指针
}VNode;

VNode graph[10];                //定义一个拥有 10 个顶点的图（邻接表法）

```

③散列表的拉链法，请参考王道书 7.5.3，数据结构示意图如下所示：

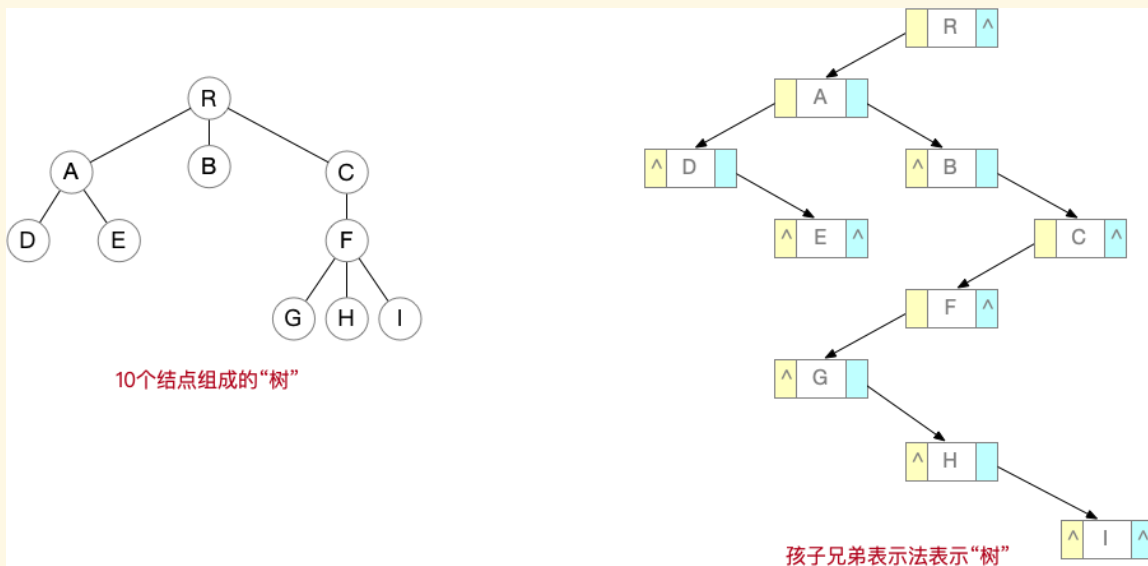


◆ 图 7.33 拉链法处理冲突的散列

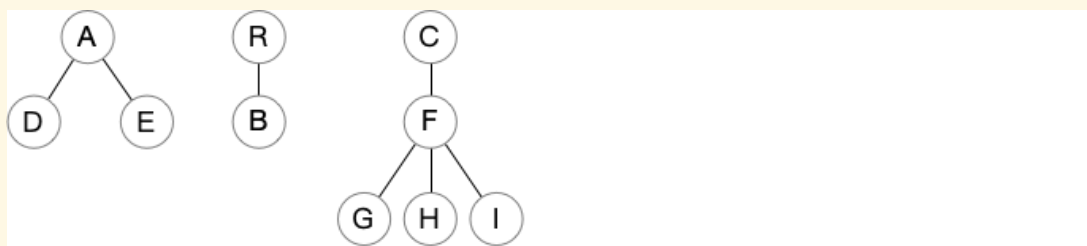
④基数排序，请参考王道书 8.5.2，算法示意图如下所示：


```
typedef struct CSNode{
    ElemType data;                                //数据域
    struct CSNode *firstchild,*nextsibling;      //第一个孩子和右兄弟指针
}CSNode,*CSTree;
```

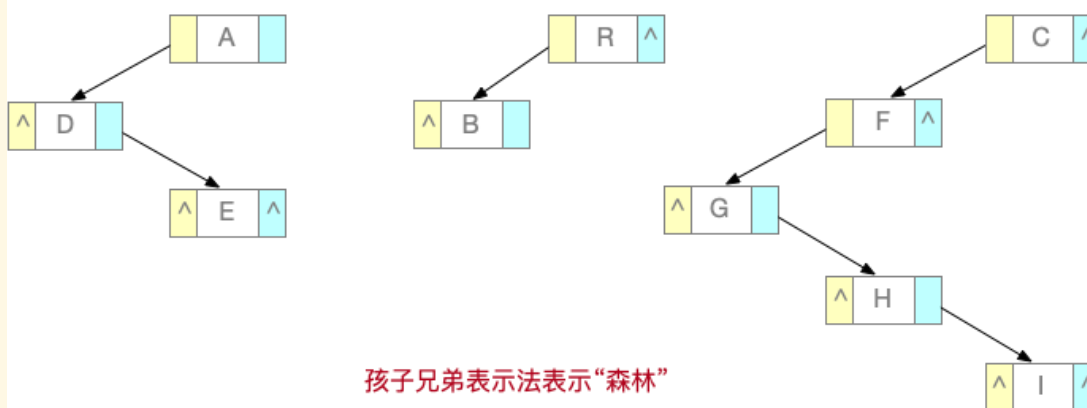
孩子表示法可表示“**多叉树**”，如下所示：



显然，**孩子表示法**可表示“森林”，如下所示：



由10个结点组成的“森林”，共计3棵树



孩子兄弟表示法表示“森林”

3.3.5 + 3.3.6 画出“树”和“森林”的数据结构示意图

请参考上面 3.3.1、3.3.2、3.3.4

特别注意：应用题中很可能考双亲表示法、孩子表示法、孩子兄弟表示法的数据结构定义+画图

3.4 哈夫曼树的应用

该部分请参考王道书、考点精讲（基础课）课件自行完成

另外，请认真完成 2020 年真题的应用题，即王道书 5.5.3 大题第 3 题

3.5 并查集的应用

3.5.1~3.5.3 实现并查集的数据结构定义，并实现 Union、Find 两个基本操作

```
#define SIZE 13

int UFsets[SIZE];          //用一个数组表示并查集

//初始化并查集
void Initial(int S[]){
    for(int i=0;i<SIZE;i++)
        S[i]=-1;
}

//Find “查”操作，找 x 所属集合（返回 x 所属根结点）
int Find(int S[],int x){
    while(S[x]>=0)          //循环寻找 x 的根
        x=S[x];
    return x;               //根的 S[] 小于 0
}

//Union “并”操作，将两个集合合并为一个
void Union(int S[],int Root1,int Root2){
    //要求 Root1 与 Root2 是不同的集合
    if(Root1==Root2) return;
    //将根 Root2 连接到另一根 Root1 下面
    S[Root2]=Root1;
}
```

可采取“小树合并到大树”的策略优化 Union 操作

```
//Union “并”操作，小树合并到大树
void Union(int S[],int Root1,int Root2){
    if(Root1==Root2) return;
    if(S[Root2]>S[Root1]) { //Root2 结点数更少
        S[Root1] += S[Root2]; //累加结点总数
        S[Root2]=Root1; //小树合并到大树
    } else {
        S[Root2] += S[Root1]; //累加结点总数
        S[Root1]=Root2; //小树合并到大树
    }
}
```

可采取“路径压缩”的策略优化 Find 操作

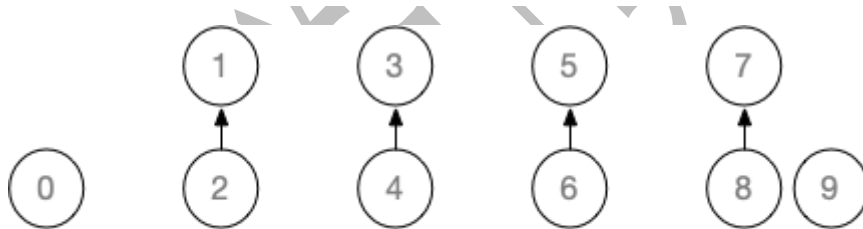
```
//Find “查”操作优化，先找到根节点，再进行“压缩路径”
int Find(int S[],int x){
    int root = x;
    while(S[root]>=0)    root=S[root]; //循环找到根
    while(x!=root){ //压缩路径
        int t=S[x];      //t 指向 x 的父节点
        S[x]=root;      //x 直接挂到根节点下
        x=t;
    }
    return root;          //返回根节点编号
}
```

3.5.4 设计一个例子，对 10 个元素 Union

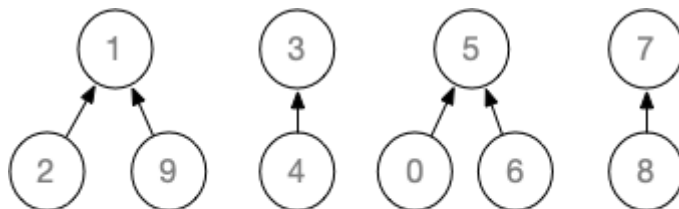
假设初始长度为 10（0 ~ 9）的并查集，按 1-2、3-4、5-6、7-8、0-5、1-9、9-3、8-0、4-6 的顺序进行 Union 操作



初始状态👉



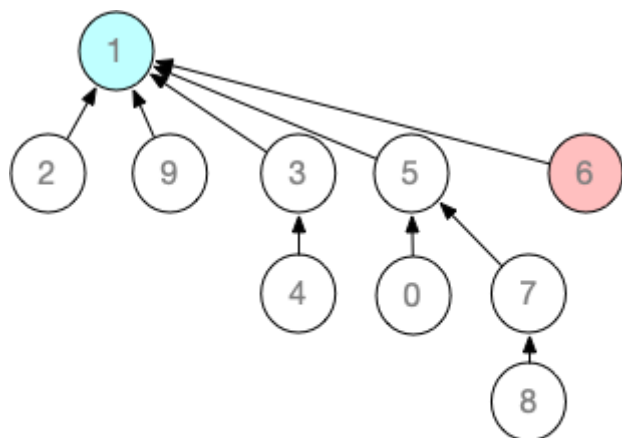
Union: 1-2、3-4、5-6、7-8👉



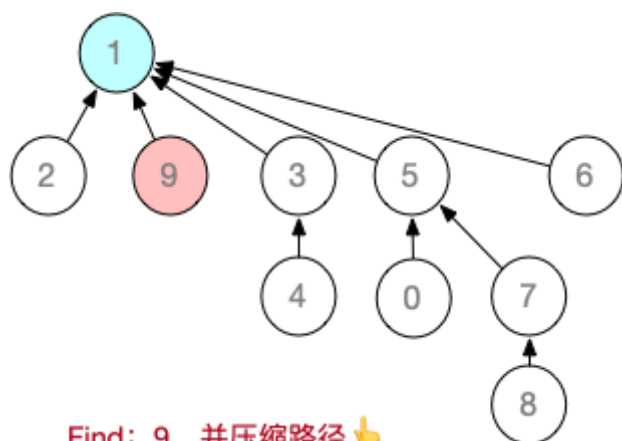
Union: 0-5、1-9👉 (小树合并到大树)

3.5.5 基于上述例子，进行若干次 Find，并完成“压缩路径”

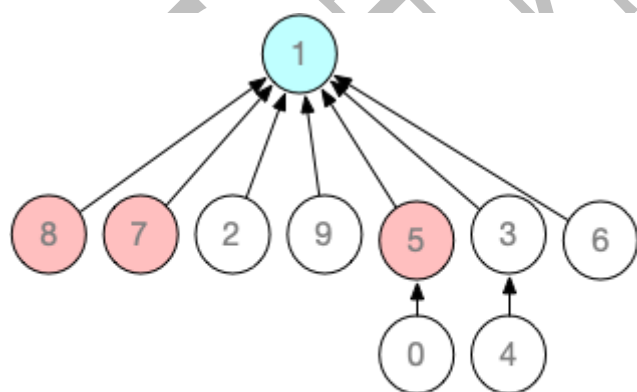
依次对 6、9、8 执行 Find 操作，并完成压缩路径



Find: 6, 并压缩路径 🖱️



Find: 9, 并压缩路径 🖱️



Find: 8, 并压缩路径 🖱️

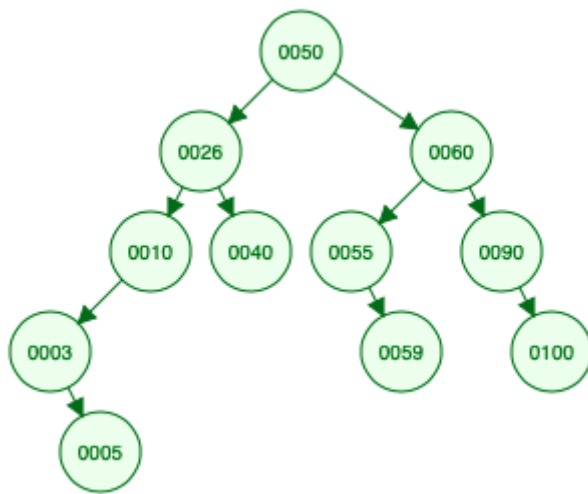
3.6 二叉排序树、平衡二叉树的应用题潜在考法

3.6.1 自己设计一个例子，给出不少于 10 个关键字序列，按顺序插入一棵初始为空的二叉排序树，画出每一次插入后的样子

注：自己设计插入序列，并在“408 快乐小网站”模拟执行

Binary Search Trees

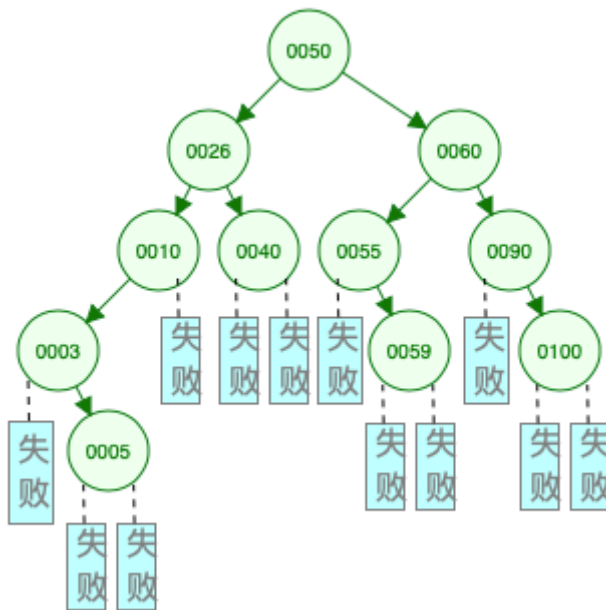
例：从一棵初始为空的 BST 开始，依次插入：50、26、10、3、5、60、90、40、55、100、59，最终结果如下



插入过程自己用网站模拟，可使用网站的“Pause、Step Forward”两个按钮单步执行。

3.6.2 基于你设计的例子，计算二叉排序树在查找成功和查找失败时的 ASL

计算“树形查找”的 ASL 时，需要补充“失败结点”，再进行计算

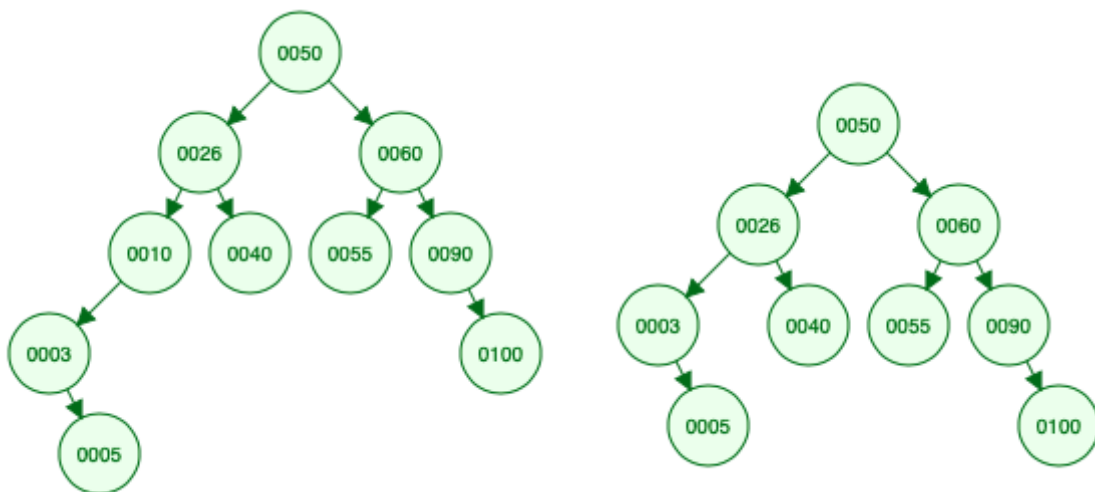


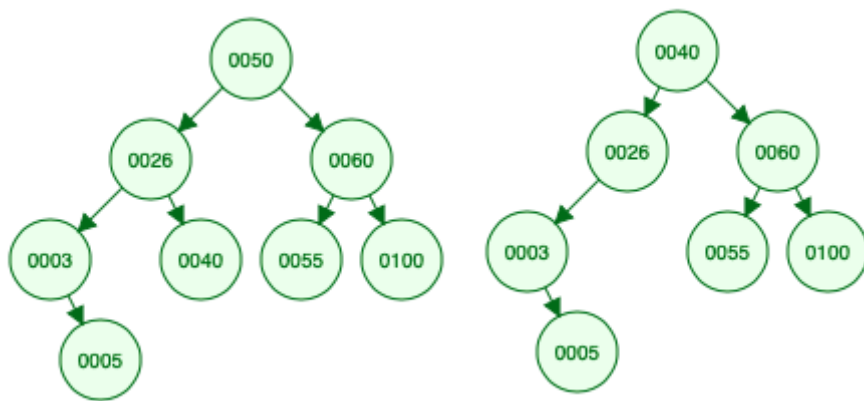
$$ASL_{成功} = \frac{1}{11} (1 \times 1 + 2 \times 2 + 3 \times 4 + 4 \times 3 + 5 \times 1) = \frac{34}{11}$$

$$ASL_{失败} = \frac{1}{12} (3 \times 5 + 4 \times 5 + 5 \times 2) = \frac{45}{12} = 3.75$$

3.6.3 基于你设计的例子，依次删除不少于 4 个元素，画出每一次删除之后的样子（需要包含四种删除情况——删一个叶子结点、删一个只有左子树的结点、删一个只有右子树的结点、删一个既有左子树又有右子树的结点）

例：基于 3.6.1 的例子，依次删除：59（叶子）、10（仅有左子树）、90（仅有右子树）、50（拥有左右子树）。4 次删除后，二叉排序树的状态分别如下：

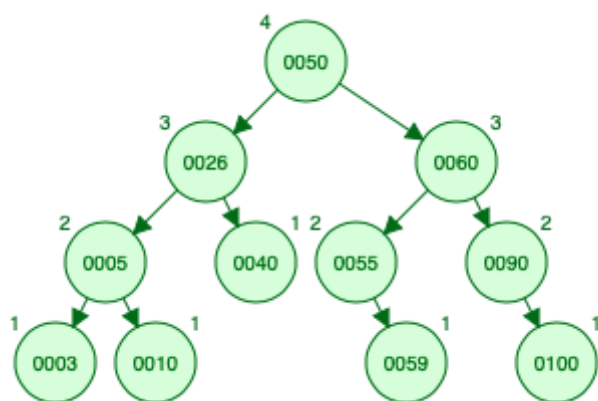




3.6.4 自己设计一个例子，给出不少于 10 个关键字序列，按顺序插入一棵初始为空的平衡二叉树，画出每一次插入后的样子（你设计的例子要涵盖 LL、RR、LR、RL 四种调整平衡的情况）

注：自己设计插入序列，并在“408 快乐小网站”模拟执行 AVL Trees (Balanced binary search trees)

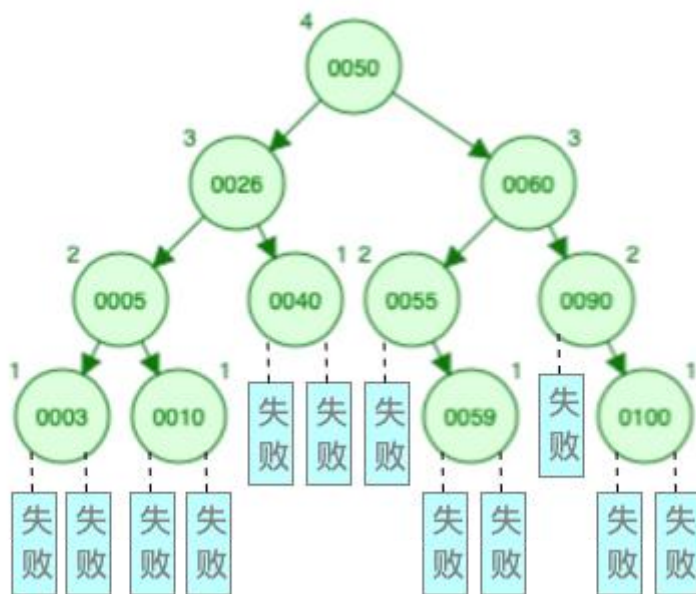
例：从一棵初始为空的 AVL Trees 开始，依次插入：50、26、10（LL）、3、5（LR）、60、90（RR）、40、55、100、59（RL），最终结果如下



插入过程自己用网站模拟，可使用网站的“Pause、Step Forward”两个按钮单步执行。上述例子中，插入元素 10、5、90、59 时分别发生了 LL、LR、RR、RL 四种“调整平衡”的情况。

3.6.5 基于你设计的例子，计算平衡二叉树在查找成功和查找失败时的 ASL

计算“树形查找”的 ASL 时，需要补充“失败结点”，再进行计算



$$ASL_{成功} = \frac{1}{11} (1 \times 1 + 2 \times 2 + 3 \times 4 + 4 \times 4) = 3$$

$$ASL_{失败} = \frac{1}{12} (3 \times 4 + 4 \times 8) = \frac{44}{12}$$

Ch4 图的应用

4.1 图的性质

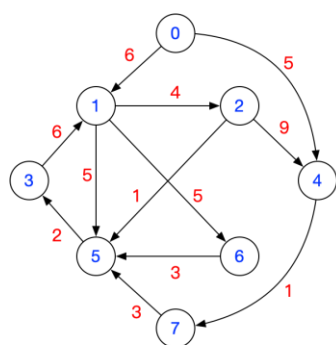
通过王道书 6.1.2 课后小题来复习 “图的性质”

4.2 图的数据结构定义

4.2.1 写代码：定义一个顺序存储的图（邻接矩阵实现）

```
#define N 8          //顶点数目的最大值，根据题目要求自己定义
typedef struct{
    char vex[N]; //N 个顶点信息，每个顶点存储一个 char 字符，可根据题目要求
                //将 char 改为其他类型
    int weight[N][N]; //N*N 邻接矩阵，每条边的权值用 int 变量表示
    int vexnum, arcnum; //图的当前顶点数和弧数
}MGraph;
```

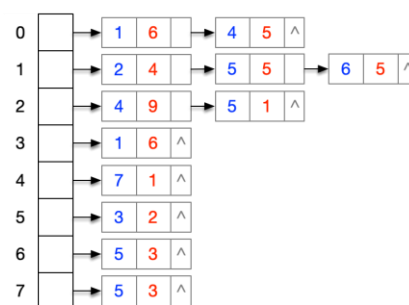
4.2.4 自己设计一个不少于 6 个结点的带权有向图，并画出其邻接矩阵、邻接表的样子



一个带权有向图

	0	1	2	3	4	5	6	7
0		6			5			
1			4			5	5	
2					9	1		
3		6						
4								1
5					2			
6						3		
7						3		

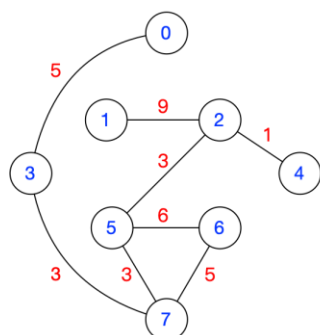
邻接矩阵



邻接表

4.3 图的应用：最小生成树

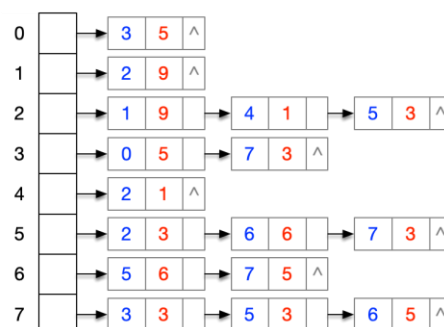
4.3.1 自己设计一个不少于 6 个结点的带权无向连通图，并画出其邻接矩阵、邻接表的样子



一个带权无向图

	0	1	2	3	4	5	6	7
0				5				
1			9					
2		9			1	3		
3	5							3
4			1					
5			3				6	3
6						6		5
7				3		3	5	

邻接矩阵（一定是对称矩阵）



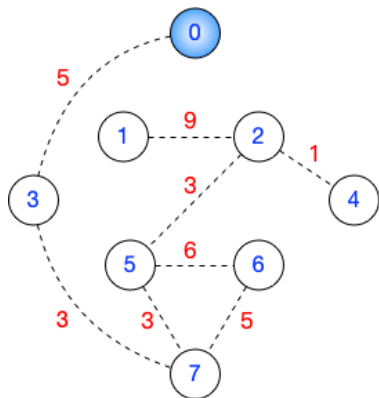
邻接表

4.3.2 基于上述无向连通图，使用 Prim 算法生成 MST，画出算法执行过程的示意图，并计算 MST 的总代价

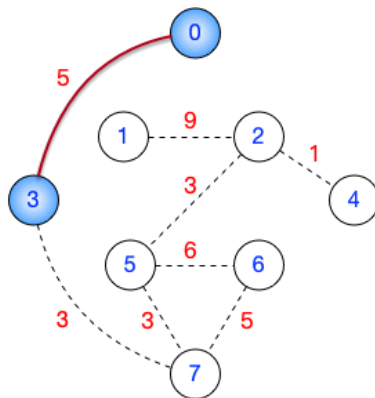
注：普利姆算法是选代价最小的“点”，克鲁斯卡尔算法是选代价最小的“边”，别记混了。

可以这么联想记忆，“Prim”是字母 P 开头的，“点”的英文是 Point，也是 P 开头的，因此 Prim 算法是选“点”。

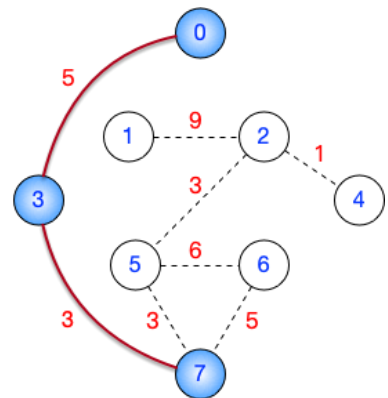
从顶点 0 开始，运行 Prim 算法的过程如下：



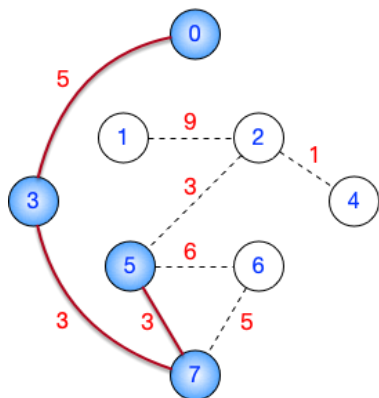
从顶点0出发



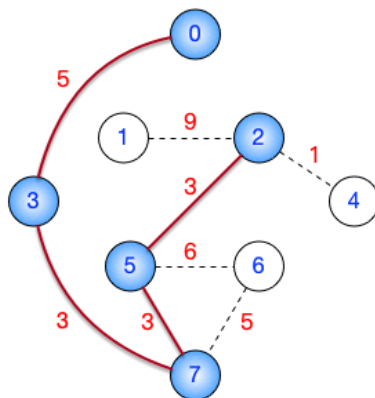
第1轮



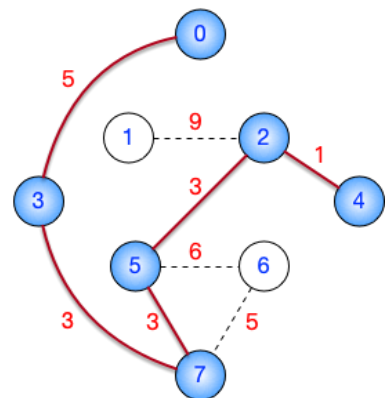
第2轮



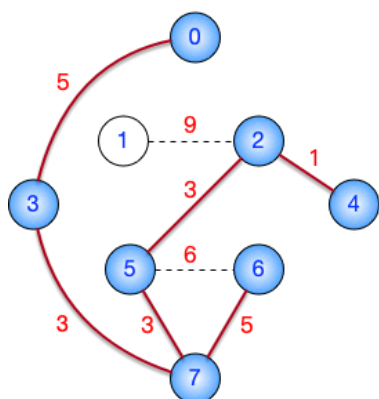
第3轮



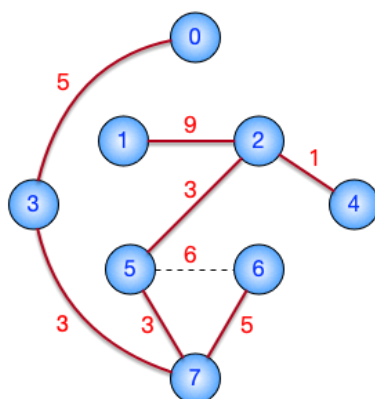
第4轮



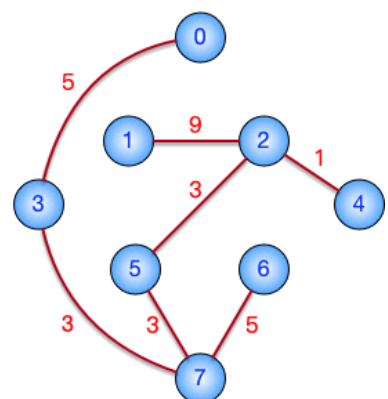
第5轮



第6轮



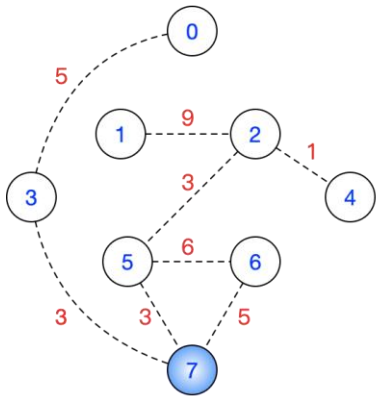
第7轮



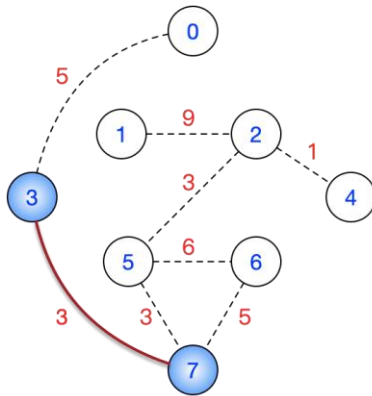
最小生成树MST

$$\text{MST总代价} = 5 + 9 + 3 + 1 + 3 + 3 + 5 = 29$$

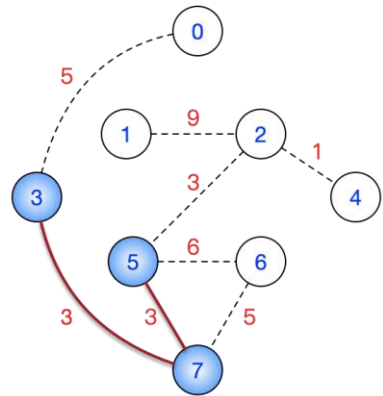
从顶点 7 开始，运行 Prim 算法的过程如下。这里想强调的是：当同时有两个代价相同的顶点时，优先将编号小的顶点加入 MST。如：第 1 轮从顶点 7 出发，与其相连的顶点 3、顶点 5 代价都是最小的，优先选择编号更小的顶点 3。



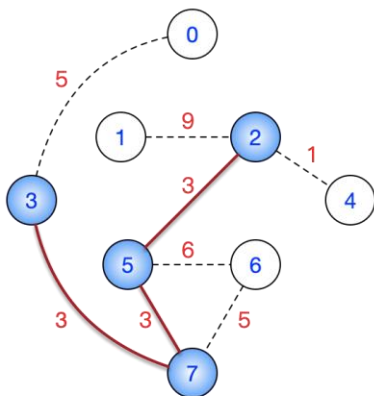
从顶点7出发



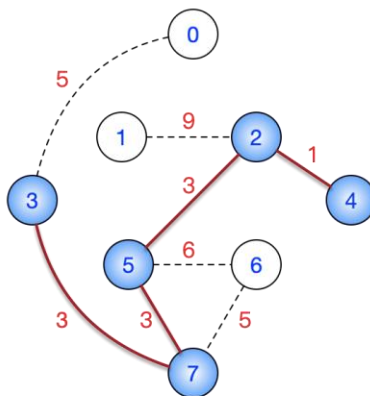
第 1 轮



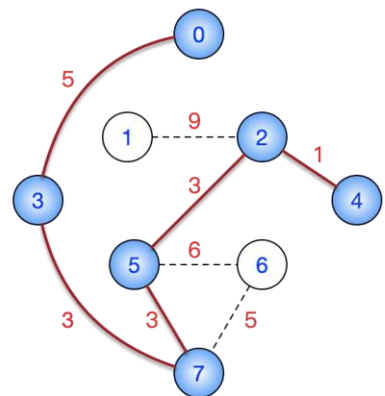
第 2 轮



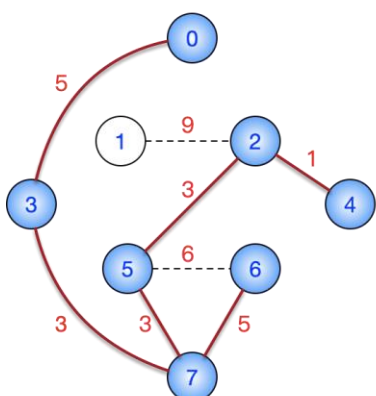
第 3 轮



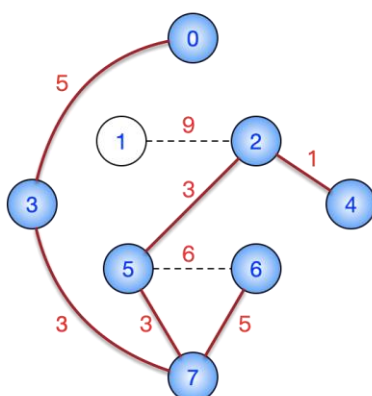
第 4 轮



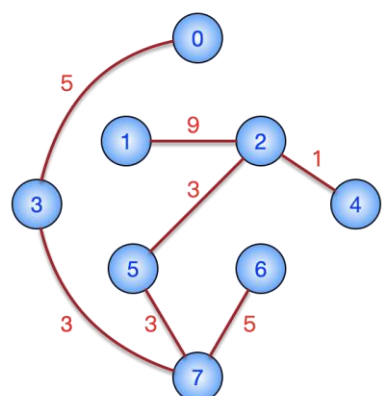
第 5 轮



第 6 轮



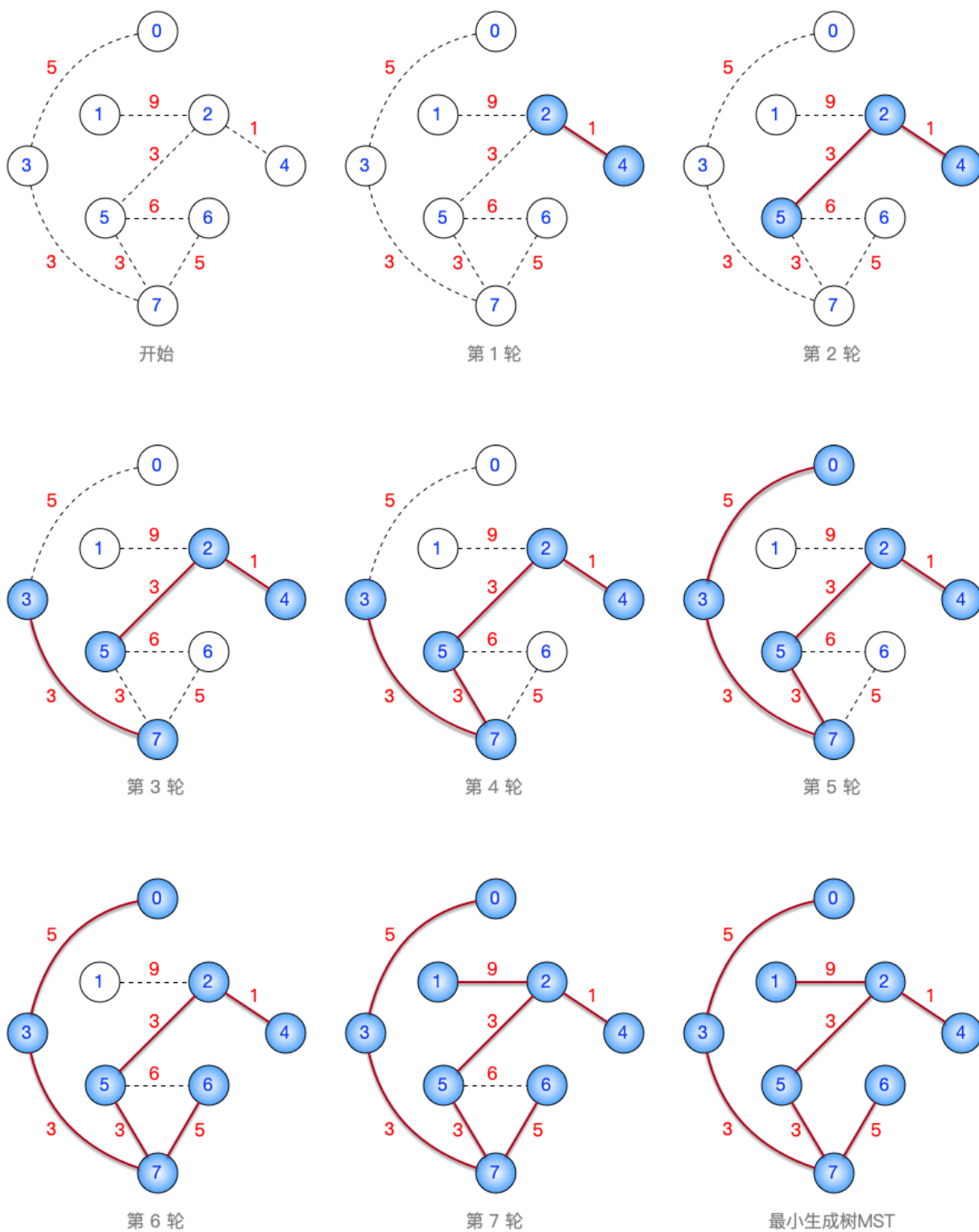
第 7 轮



最小生成树MST

$$\text{MST总代价} = 5 + 9 + 3 + 1 + 3 + 3 + 5 = 29$$

4.3.3 基于上述无向连通图，使用 Kruskal 算法生成 MST，画出算法执行过程的示意图，并计算 MST 的总代价



$$\text{MST总代价} = 5 + 9 + 3 + 1 + 3 + 3 + 5 = 29$$

Kruskal 算法的执行过程是：先将各条边按照权值递减的排序，再按顺序依次检查是否将这些边加入 MST。各条边排序结果如下：

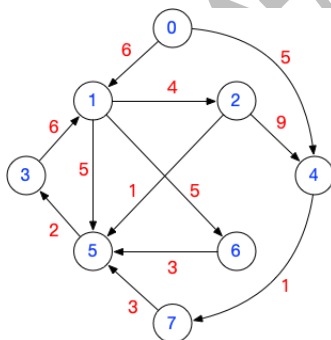
(2,4) —— 权值 1
 (2,5) —— 权值 3
 (3,7) —— 权值 3
 (5,7) —— 权值 3
 (0,3) —— 权值 5
 (6,7) —— 权值 5
 (5,6) —— 权值 6
 (1,2) —— 权值 9

可见，当多条边的权值相同时，“起点”编号更小的边排在前面。因此，在 Kruskal 算法的第 2 轮中，虽然有 3 条边 (2,5) (3,7) (5,7) 的权值都是最小的，但算法会优先选择将排序靠前的 (2,5) 加入 MST。

4.4 图的应用：最短路径

4.4.1 基于你设计的**带权有向图**，从某一结点出发，执行 Dijkstra 算法求单源最短路径。用文字描述每一轮执行的过程

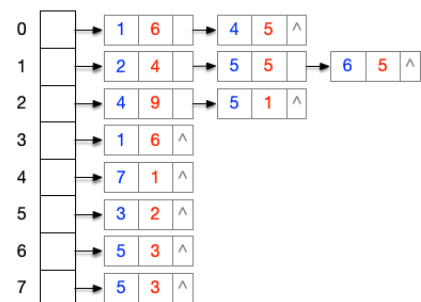
下面将**从顶点 1 出发**，执行 Dijkstra 算法



一个带权有向图

	0	1	2	3	4	5	6	7
0		6			5			
1			4			5	5	
2								
3								
4								1
5					2			
6						3		
7						3		

邻接矩阵



邻接表

注 1：如果题目让你描述迪杰斯特拉算法的执行过程，可以试着模仿王道书画一张表。如下所示。你会发现这个表画起来很崩溃，复杂的飞起。相信我，老师改卷也不愿意改这么复杂的答案。因此，考场上不太可能让你画这么复杂的表。

顶点（这一列不包含起点）	第 1 轮	第 2 轮	第 3 轮	第 4 轮	第 5 轮	第 6 轮	第 7 轮
0	∞	∞	∞	∞	∞	∞	∞
2	4 1→2	已完成	已完成	已完成	已完成	已完成	已完成
3	∞	∞	7 1→5→3	7 1→5→3	已完成	已完成	已完成
4	∞	13 1→2→4	13 1→2→4	13 1→2→4	13 1→2→4	已完成	已完成
5	5 1→5	5 1→5	已完成	已完成	已完成	已完成	已完成
6	5 1→6	5 1→6	5 1→6	已完成	已完成	已完成	已完成
7	∞	∞	∞	∞	∞	14 1→2→4 →7	已完成
集合 S	{1, 2}	{1, 2, 5}	{1, 2, 5, 6}	{1, 2, 5, 6, 3}	{1, 2, 5, 6, 3, 4}	{1, 2, 5, 6, 3, 4, 7}	{1, 2, 5, 6, 3, 4, 7, 0}

注 2：以下是一种更适合考试的“简洁答题方法”。自己打草稿，快速确定每一轮 Dijkstra 算法的运行结果，然后按下面这种方式答题到试卷上。

答：

从顶点 1 出发，运行迪杰斯特拉算法的过程如下：

第一轮：顶点 1 到 2 的最短路径为 $1 \rightarrow 2$ ，距离为 4

第二轮：顶点 1 到 5 的最短路径为 $1 \rightarrow 5$ ，距离为 5

第三轮：顶点 1 到 6 的最短路径为 $1 \rightarrow 6$ ，距离为 5

第四轮：顶点 1 到 3 的最短路径为 $1 \rightarrow 5 \rightarrow 3$ ，距离为 7

第五轮：顶点 1 到 4 的最短路径为 $1 \rightarrow 2 \rightarrow 4$ ，距离为 13

第六轮：顶点 1 到 7 的最短路径为 $1 \rightarrow 2 \rightarrow 4 \rightarrow 7$ ，距离为 14

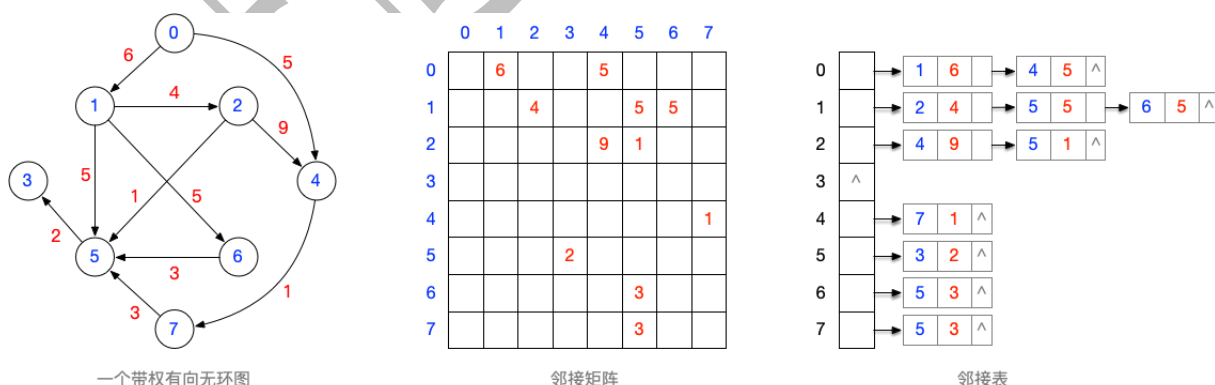
第七轮：不存在顶点 1 到 0 的路径，距离为 ∞

4.4.2 文字描述：用 BFS 算法求单源最短路径的过程

注意：BFS 算法只能对无权图求最短路径

4.5 图的应用：拓扑排序

4.5.1 自己设计一个不少于 6 个结点的带权有向无环图，并画出其邻接矩阵的样子



4.5.2 用一维数组将你设计的有向无环图的邻接矩阵进行压缩存储

注：有向无环图，一定可以转化为一个上三角或下三角矩阵。但是需要调整顶点的编号。

如果要用上三角矩阵表示有向无环图的邻接矩阵，可以对图进行拓扑排序，按照拓扑排序序

“上三角矩阵”可以按行优先压缩存储，如下所示：

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	
	6		5						4		5	5				9			1					1				3			3				2	

上三角矩阵的压缩存储

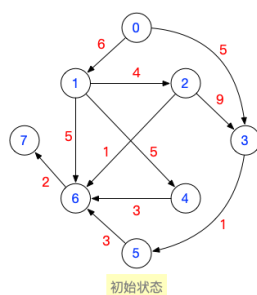
4.5.3 文字描述：基于你压缩存储的数组，如何判断结点 i、j 之间是否有边？

行优先压缩存储上三角矩阵，元素 (i, j) 和一维数组间的对应关系为：

$$k = \frac{(i-1)(2n-i+2)}{2} + (j-i), i \leq j \quad (\text{上三角区和主对角线元素})$$

只需用上述公式将 i、j 转化为 k，再检查一维数组中下标为 k 的元素即可。

4.5.4 基于你设计的带权有向无环图，写出所有合法的拓扑排序序列



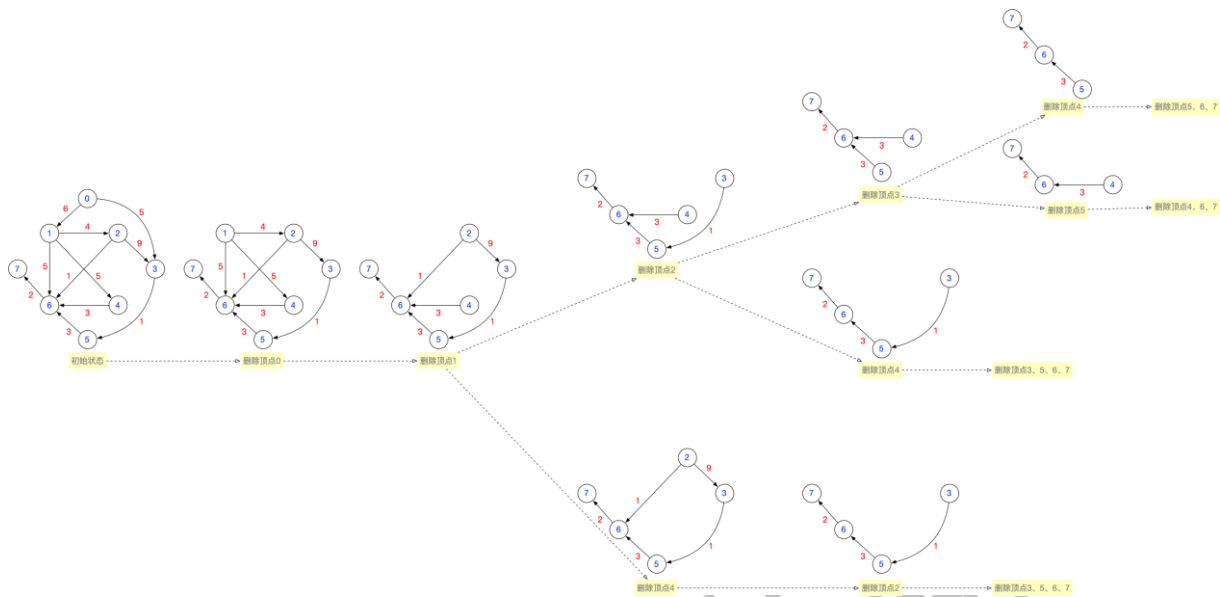
拓扑排序序列 1：0,1,2,3,4,5,6,7

拓扑排序序列 2：0,1,2,3,5,4,6,7

拓扑排序序列 3：0,1,2,4,3,5,6,7

拓扑排序序列 4：0,1,4,2,3,5,6,7

可参考下面这张图分析所有可能出现的拓扑排序序列：



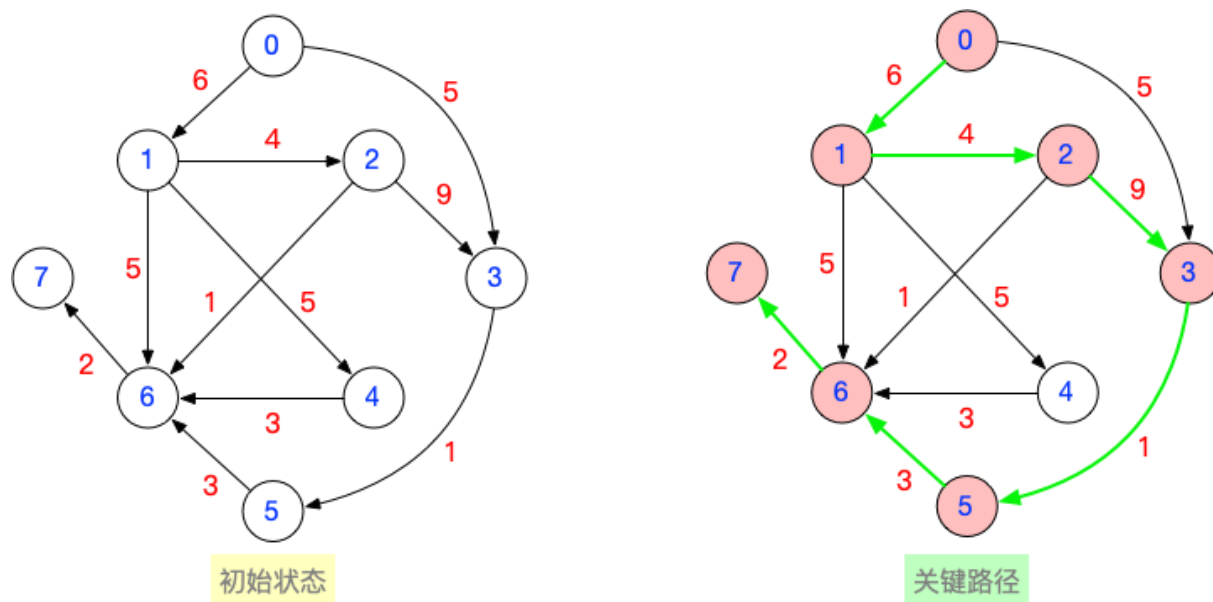
4.5.5 文字描述：拓扑排序的过程

- 拓扑排序
- ① 从AOV网中选择一个没有前驱（入度为0）的顶点并输出
 - ② 从网中删除该顶点和所有以它为起点的有向边
 - ③ 重复①和②直到当前的AOV网为空

- 逆拓扑排序
- ① 从AOV网中选择一个没有后继（出度为0）的顶点并输出
 - ② 从网中删除该顶点和所有以它为终点的有向边
 - ③ 重复①和②直到当前的AOV网为空

4.6 图的应用：关键路径

4.6.1 基于你设计的带权有向无环图，写出所有合法的关键路径，并算出关键路径总长度

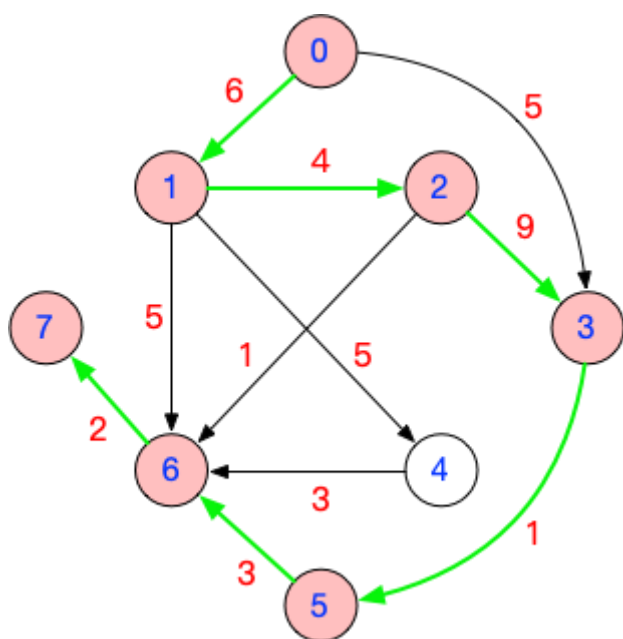


用手算的方式求关键路径，就是要找到从起点到终点的所有路径中最长的那条。

在上面这个图中，起点是 0，终点是 7，从 0 到 7 最长的路径是 $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 7$ ，关键路径总长度为 25

4.6.2 文字描述：关键路径总长度的现实意义是什么？

AOE 网可以表示一个项目，每个**顶点表示事件**，每个**有向边表示活动**。关键路径的总长度为 25，意味着从启动项目到完成该项目，时间开销至少需要 25。



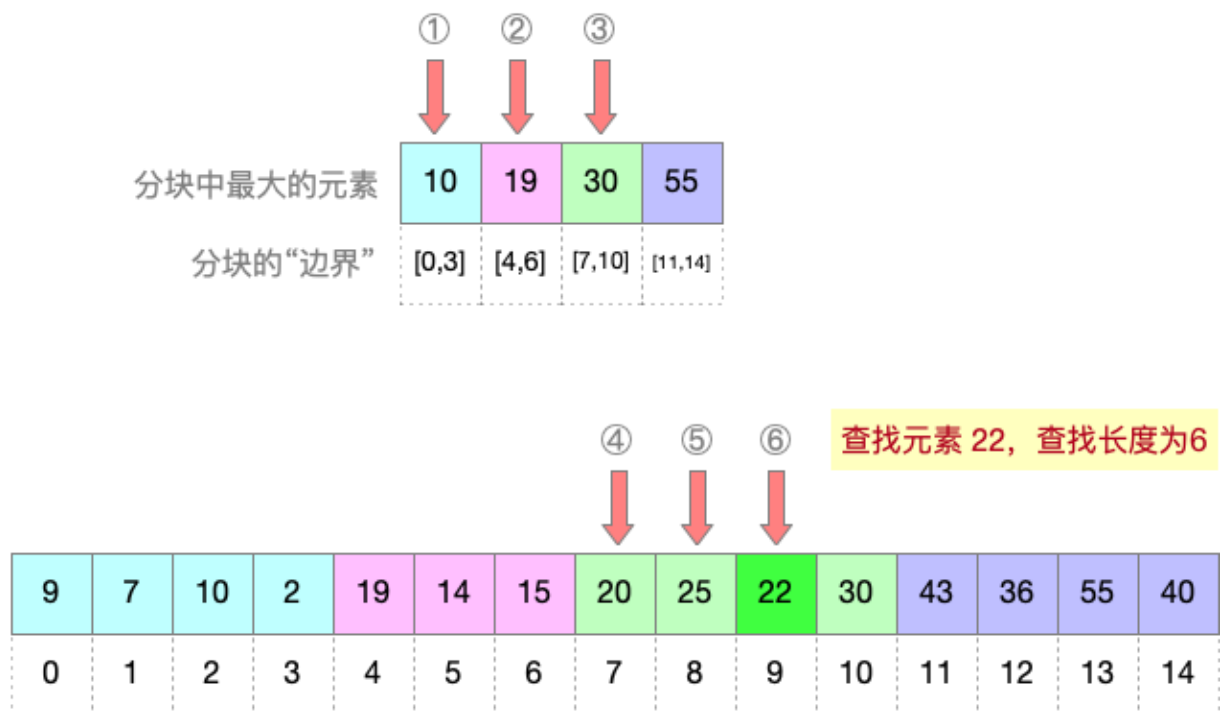
Ch5 查找算法的分析和应用

考研大纲中，要求我们掌握的查找算法是：

1. 顺序查找
2. 分块查找
3. 折半查找
4. 树形查找（二叉查找树、平衡二叉树、红黑树）
5. B 树
6. 散列查找
7. 字符串的模式匹配

5.1 分块查找

5.1.1 自己设计一个分块查找的例子，不少于 15 个数据元素，并建立分块查找的索引



5.1.2 基于上述例子，计算查找成功的 ASL、查找失败的 ASL

上图中，共有 15 个元素，计算查找成功时的平均查找长度 ASL，每个元素被查找的概率相同，都是 $1/15$ 。每个元素的查找都需要先查分块索引，再顺序查找分块内各个元素。

查找元素 9 时，查找长度为 $1+1$

查找元素 7 时，查找长度为 $1+2$

查找元素 10 时，查找长度为 $1+3$

查找元素 2 时，查找长度为 $1+4$

查找元素 19 时，查找长度为 $2+1$

查找元素 14 时，查找长度为 $2+2$

查找元素 15 时，查找长度为 $2+3$

查找元素 20 时，查找长度为 $3+1$

查找元素 25 时，查找长度为 $3+2$

查找元素 22 时，查找长度为 $3+3$

查找元素 30 时，查找长度为 $3+4$

查找元素 43 时，查找长度为 $4+1$

查找元素 36 时，查找长度为 4+2

查找元素 55 时，查找长度为 4+3

查找元素 40 时，查找长度为 4+4

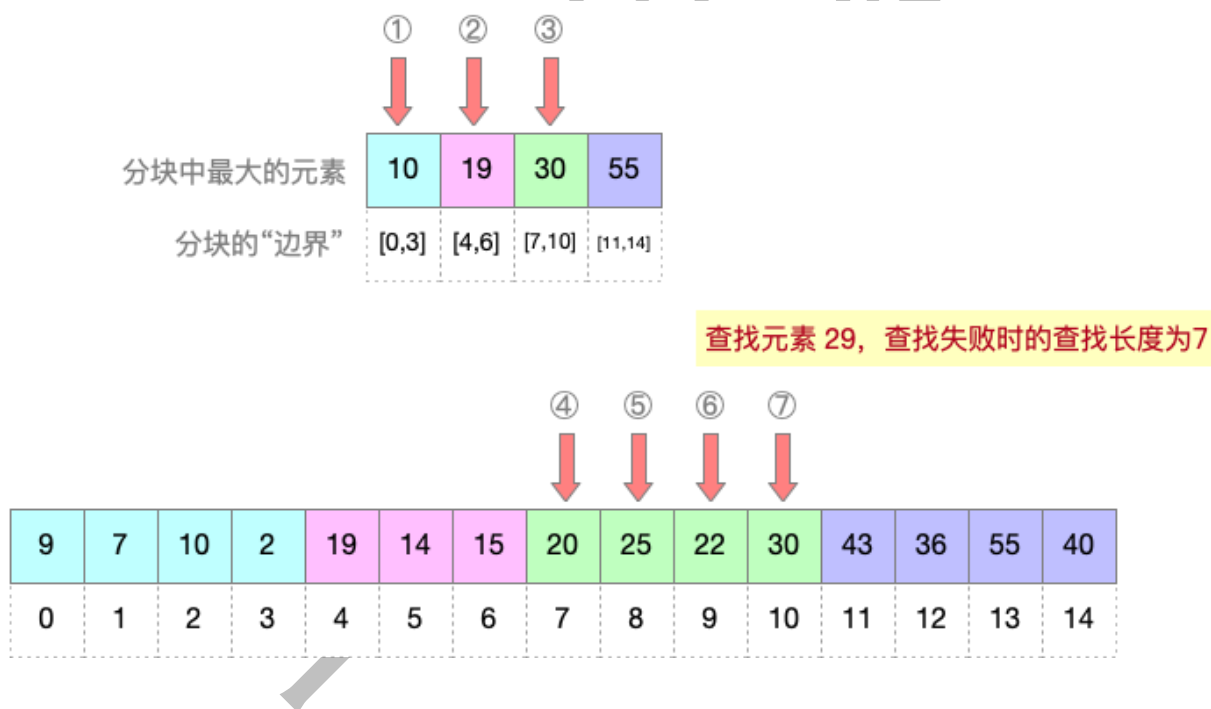
综上，查找成功的 ASL = $(2+3+4+5+3+4+5+4+5+6+7+5+6+7+8) \times 1/15 = 4.933$

对于查找失败的情况，题目不太可能让你分析查找失败的平均查找长度 ASL（因为查找失败的情形很多，我们无法预估出现每一种失败情形的概率）。

因此，如果分块查找要考察“查找失败”的情形，很有可能是给你一个特定的关键字，让你分析该关键字查找失败时的查找长度。

例：如下图所示，查找元素 29（查找失败）时，首先顺序查找分块索引；确认元素 29 属于第三个分块，该分块在数组中下标范围是 7~10；紧接着在数组中查找下标为 7~10 的几个元素，所有元素和目标关键字都不匹配，最终可确定查找失败。

综上，查找元素 29（查找失败）时，总计对比关键字 7 次。



5.2 折半查找

注 1：关于折半查找的算法效率分析，可参考基础课课件，重点复习“如何画查找分析树”、“如何计算查找成功、查找失败的 ASL”

注 2：关于折半查找的算法执行过程，可以去 408 快乐小网站玩一玩，传送门如下：

<https://www.cs.usfca.edu/~galles/visualization/Search.html>

Searching Sorted List

591 Linear Search Binary Search ☒ Small ☐ Large

```
def binarySearch(listData, value)
low = 0
high = len(listData) - 1
while (low <= high)
mid = (low + high) / 2
if (listData[mid] == value):
return mid
elif (listData[mid] < value)
low = mid + 1
else:
high = mid - 1
return -1
```

点这里，折半查找

Searching For 591 Result 24 Element found

low 24 mid 24 high 24

12	21	44	100	130	214	235	246	273	278	298	331	347	350	351	364
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

417	422	424	456	473	514	540	566	591	622	650	652	680	815	859	932
16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31

Animation Completed

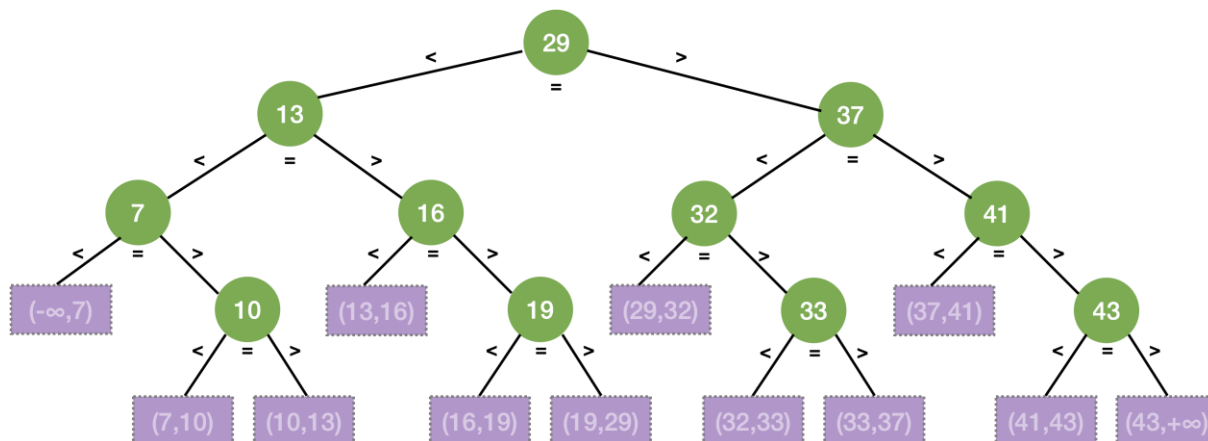
Skip Back Step Back Pause Step Forward Skip Forward w: 1000 h: 500 Change Canvas Size Move Controls

Animation Speed

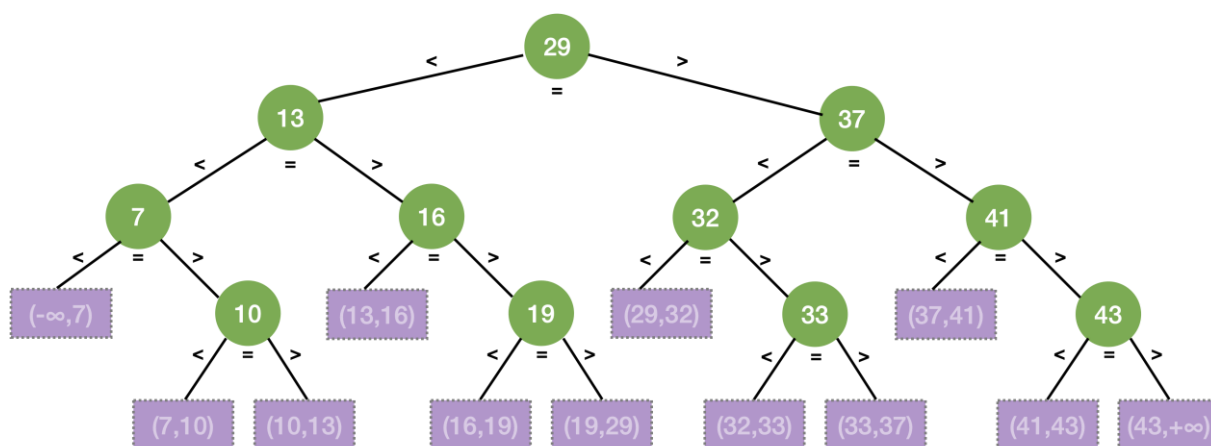
Algorithm Visualizations

5.2.1 自己设计一个折半查找的例子，不少于 10 个数据元素，画出对应的查找分析树

7	10	13	16	19	29	32	33	37	41	43
0	1	2	3	4	5	6	7	8	9	10



5.2.2 基于上述例子，计算查找成功的 ASL、查找失败的 ASL



$$ASL_{成功} = (1*1 + 2*2 + 3*4 + 4*4) / 11 = 3$$

$$ASL_{失败} = (3*4 + 4*8) / 12 = 11/3$$

5.3 散列查找

5.3.1 自己设计一个散列表，总长度由你决定，并设计一个合理的散列函数，使用线性探测法解决冲突

设散列表长度为 16，采用线性探测法解决冲突，从空表开始，依次插入关键字 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key) = key \% 13$

注：散列表的长度可以比散列函数的映射范围更大，散列函数的映射范围是 0~12，散列表的后面几个位置 13~15 不可能被散列函数直接映射，但是在线性探测法处理冲突时，可能被使用到

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

5.3.2 基于上述散列表，设计不少于 10 个元素的插入序列，依次插入散列表，画出散列表最终的样子（插入过程至少发生 4 次冲突）

$19 \% 13 = 6$ ，位置#6 未发生冲突，插入元素 19

$14 \% 13 = 1$ ，位置#1 未发生冲突，插入元素 14

$23 \% 13 = 10$ ，位置#10 未发生冲突，插入元素 23

	14					19				23					
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

$1 \% 13 = 1$ ，位置#1 发生冲突，采用线性探测法解决冲突，最终在位置#2 插入元素 1



68%13=3, 位置#3 未发生冲突, 插入元素 68

20%13=7, 位置#7 未发生冲突, 插入元素 20



84%13=6, 位置#6 发生冲突, 采用线性探测法解决冲突, 最终在位置#8 插入元素 84



27%13=1, 位置#1 发生冲突, 采用线性探测法解决冲突, 最终在位置#4 插入元素 27



55%13=3, 位置#3 发生冲突, 采用线性探测法解决冲突, 最终在位置#5 插入元素 55



11%13=11, 位置#11 未发生冲突, 插入元素 11



10%13=10, 位置#10 发生冲突, 采用线性探测法解决冲突, 最终在位置#12 插入元素 10



79%13=1, 位置#1 发生冲突, 采用线性探测法解决冲突, 最终在位置#9 插入元素 79



5.3.3 基于上述例子，计算查找成功的 ASL、查找失败 ASL

考虑查找成功的所有情况。各元素查找成功时的查找长度如下：

19%13=6——1次 27%13=1——4次 55%13=3——3次
14%13=1——1次 68%13=3——1次 11%13=11——1次
23%13=10——1次 20%13=7——1次 10%13=10——3次
1%13=1——2次 84%13=6——3次 79%13=1——9次

$$ASL_{成功} = \frac{1 + 1 + 1 + 2 + 4 + 1 + 1 + 3 + 3 + 1 + 3 + 9}{12} = 2.5$$

考虑查找失败的所有情况。若目标元素的散列值为 0，则只需对比位置#0 的关键字即可确定查找失败，查找长度为 1；若目标元素的散列值为 1，采用线性探测法，需对比位置#1~#13，才能确定查找失败，查找长度是 13；若目标元素的散列值为 2，采用线性探测法，需对比位置#2~#13，才能确定查找失败，查找长度是 12；其他散列值同理。

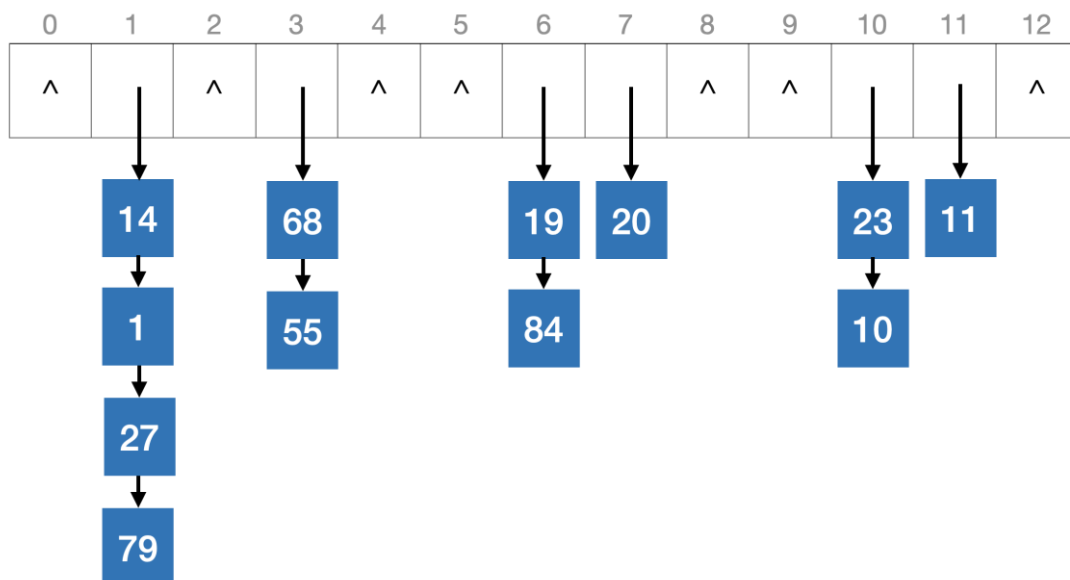
$$ASL_{失败} = \frac{1 + 13 + 12 + 11 + 10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2}{13} = 7$$

5.3.4 自己设计一个散列表，总长度由你决定，并设计一个合理的散列函数，使用拉链法解决冲突

设散列表长度为 13，采用拉链法解决冲突，从空表开始，依次插入关键字 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key)=key\%13$

注：如果采用拉链法处理冲突，则散列表的长度应该和散列函数的映射范围相同，散列函数的映射范围是 0~12。

5.3.5 基于上述散列表，设计不少于 10 个元素的插入序列，依次插入散列表，画出散列表最终的样子（插入过程至少发生 4 次冲突）



用**拉链法**（又称链接法、链地址法）处理“冲突”：**把所有“同义词”存储在一个链表中**

5.3.6 基于上述例子，计算查找成功的 ASL、查找失败的 ASL

考虑查找成功的所有情况。元素 14、68、19、29、23、11 的查找长度为 1；元素 1、55、84、10 的查找长度为 2；元素 27 的查找长度为 3；元素 79 的查找长度为 4，因此：

$$ASL_{成功} = \frac{1 * 6 + 2 * 4 + 3 + 4}{12} = 1.75$$

考虑查找失败的所有情况。若目标元素的散列值为 0，则无需对比任何关键字即可确定查找失败，查找长度为 0；若目标元素的散列值为 1，则需要依次对比关键字 14、1、27、79 才能确定查找失败，查找长度为 4；其他散列值同理。

$$ASL_{失败} = \frac{0 + 4 + 0 + 2 + 0 + 0 + 2 + 1 + 0 + 0 + 2 + 1 + 0}{13} = 0.92$$

Ch6 排序算法的分析和应用

“排序算法”部分，在应用题中最有可能考察的算法，应该具备如下特性：

1. 算法的代码比较复杂，不适合考代码（如 堆排序）
2. 算法不能太简单，太简单的算法在选择题中考察即可（如：插入排序、选择排序、冒泡排序）

因此，在内部排序算法中，应用题部分应该重点关注：希尔排序、堆排序、基数排序。另外，快速排序通常在算法题中考察，不过仍然建议大家画图梳理一遍的快排算法的执行过程。

6.1 希尔排序

6.1.1 自己设计一个长度不小于 10 的乱序数组，用希尔排序，自己设定希尔排序参数

6.1.2 画出每一轮希尔排序的状态

希尔排序示例👉

初始状态
总长度=15

36	18	10	2	88	53	27	99	0	38	46	77	78	55	40
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

第一趟： $d_1=15/2=7$
相同颜色为同一分组

36	18	10	2	88	53	27	99	0	38	46	77	78	55	40
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

第一趟排序结果

36	0	10	2	77	53	27	40	18	38	46	88	78	55	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

第二趟： $d_2=d_1/2=3$
相同颜色为同一分组

36	0	10	2	77	53	27	40	18	38	46	88	78	55	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

第二趟排序结果

2	0	10	27	40	18	36	46	53	38	55	88	78	77	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

第三趟： $d_3=d_2/2=1$
相同颜色为同一分组

2	0	10	27	40	18	36	46	53	38	55	88	78	77	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

第三趟排序结果

0	2	10	18	27	36	38	40	46	53	55	77	78	88	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

6.2 堆排序

6.2.1 自己设计一个长度不小于 10 的乱序数组，用堆排序，最终要生成升序数组，画出建堆后的状态

假设乱序数组的初始状态如下，元素从 0 开始存储：

初始状态 总长度=15	36	18	10	2	88	53	27	99	0	38	46	77	78	55	40
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

注：参考本文档 3.1.3~3.1.5

若顺序二叉树从数组下标 1 开始存储结点，则：

- 结点 i 的父结点编号为 $i/2$
- 结点 i 的左孩子编号为 $i*2$
- 结点 i 的右孩子编号为 $i*2+1$

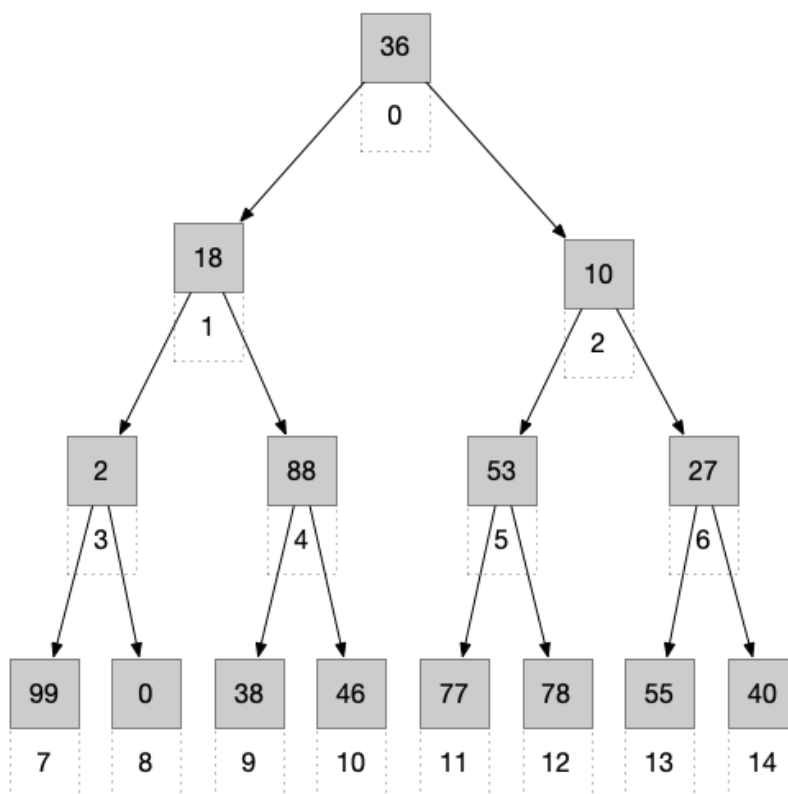
若顺序二叉树从数组下标 0 开始存储结点，则：

- 结点 i 的父结点编号为 $[(i+1)/2] - 1$
- 结点 i 的左孩子编号为 $[(i+1)*2] - 1 = 2*i + 1$
- 结点 i 的右孩子编号为 $[(i+1)*2+1] - 1 = 2*i + 2$

在本例中，元素从数组下标 0 开始存储，因此，0 号元素是根节点，1 号元素是其左孩子，2 号元素是其右孩子。其他元素间的关系如下：

初始状态
总长度=15

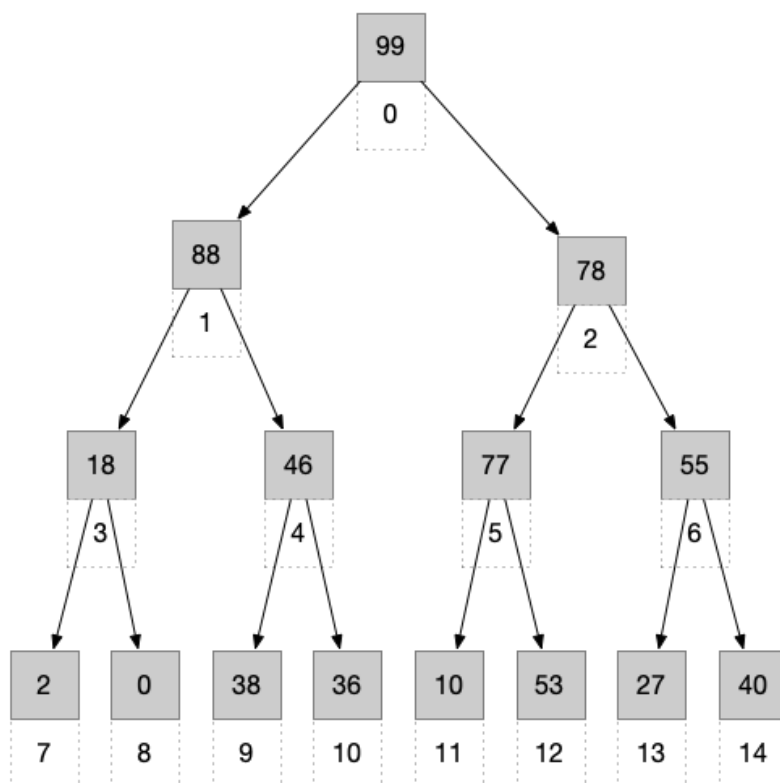
36	18	10	2	88	53	27	99	0	38	46	77	78	55	40
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



由于最终要生成升序数组，因此需要建立大根堆，从最后一个分支（即 6 号结点）开始调整，即依次调整结点 6、5、4、3、2、1、0。建立好的大根堆如下：

完成建堆
(大根堆)

99	88	78	18	46	77	55	2	0	38	36	10	53	27	40
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

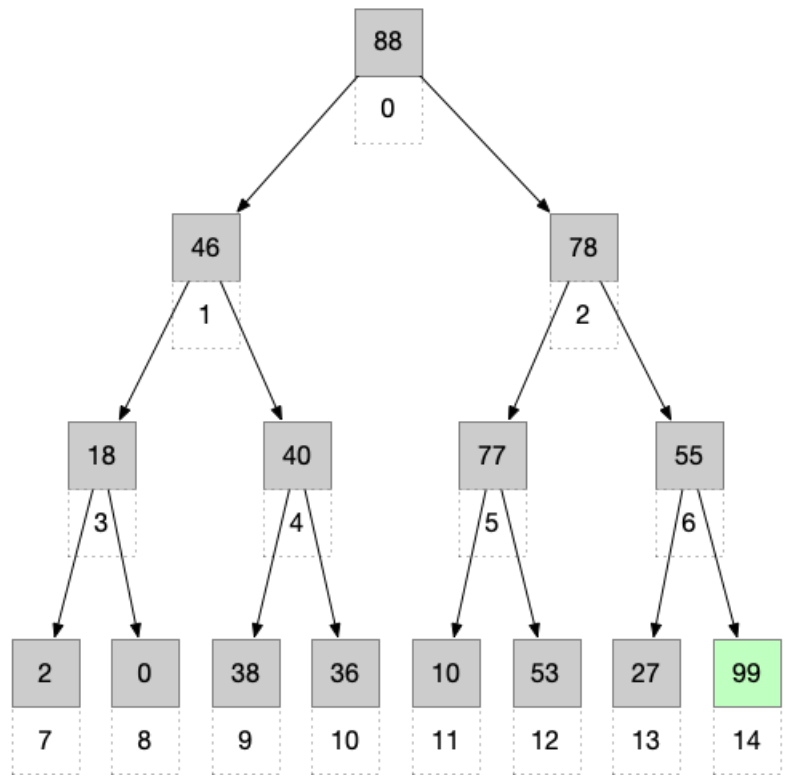


注：如果应用题让你画出一个乱序数组建堆后的样子，只需要画出数组形式的图示即可，不用画二叉树形态的图示。如下

初始状态 总长度=15	36	18	10	2	88	53	27	99	0	38	46	77	78	55	40
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
↓ 建堆															
完成建堆 (大根堆)	99	88	78	18	46	77	55	2	0	38	36	10	53	27	40
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

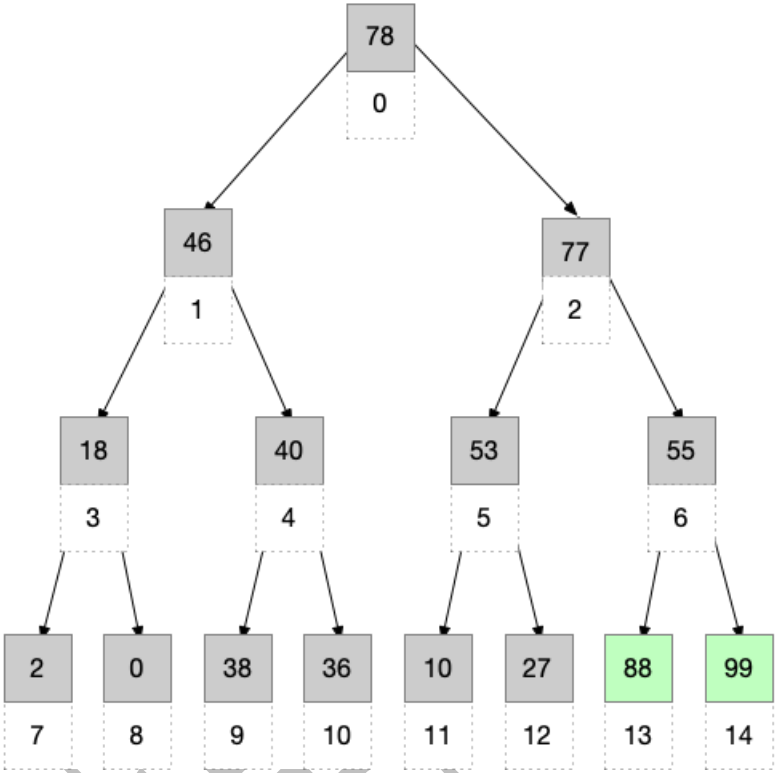
6.2.2 画出每一轮堆排序的状态

第一轮排序:	88	46	78	18	40	77	55	2	0	38	36	10	53	27	99
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



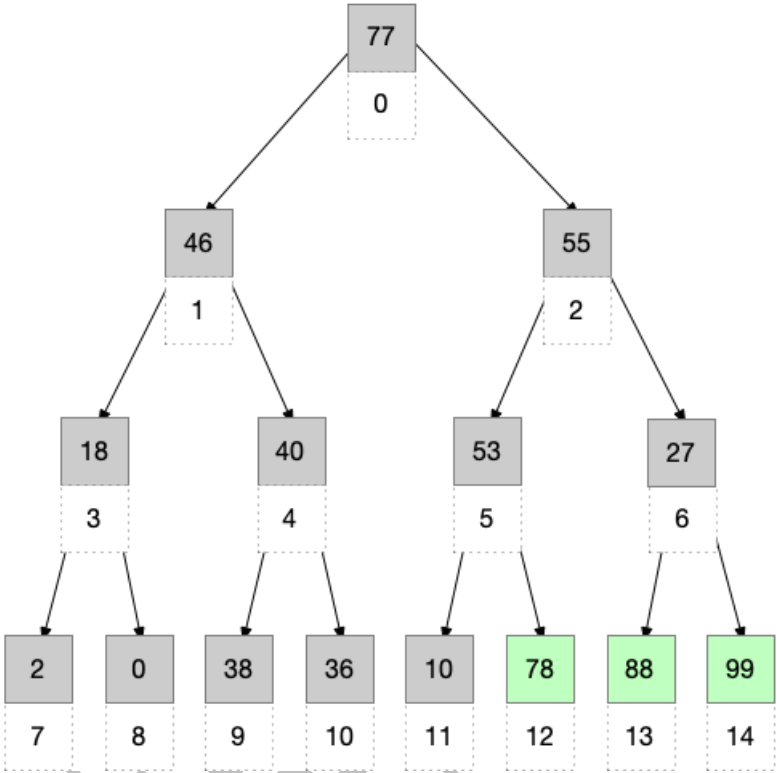
第二轮排序:

78	46	77	18	40	53	55	2	0	38	36	10	27	88	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



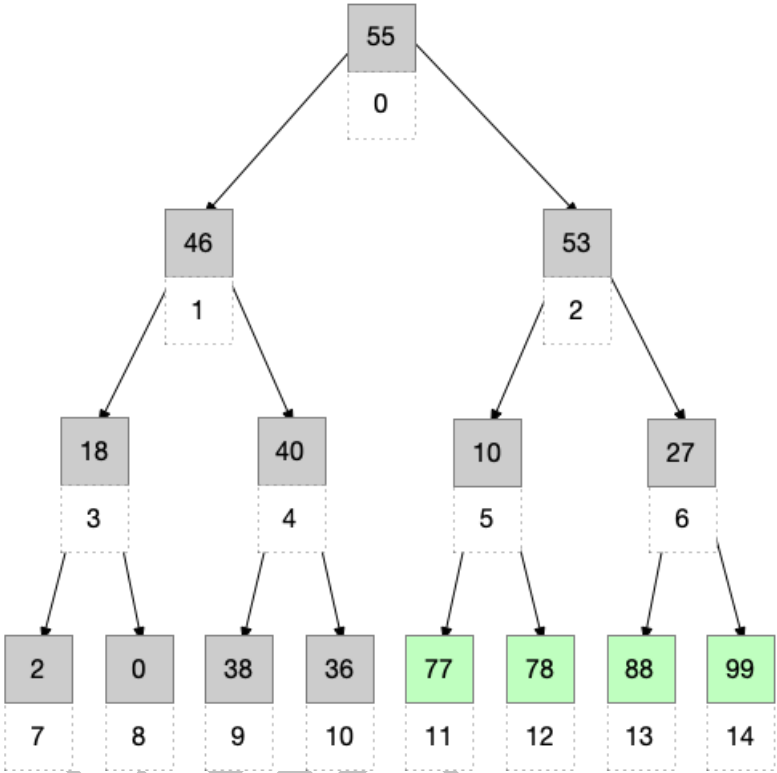
第三轮排序:

77	46	55	18	40	53	27	2	0	38	36	10	78	88	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



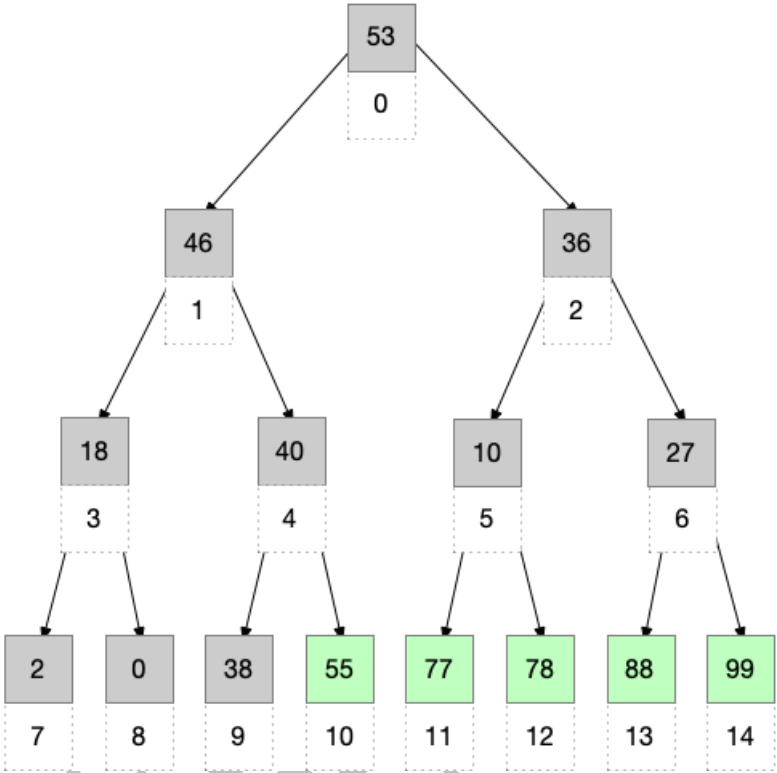
第四轮排序:

55	46	53	18	40	10	27	2	0	38	36	77	78	88	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



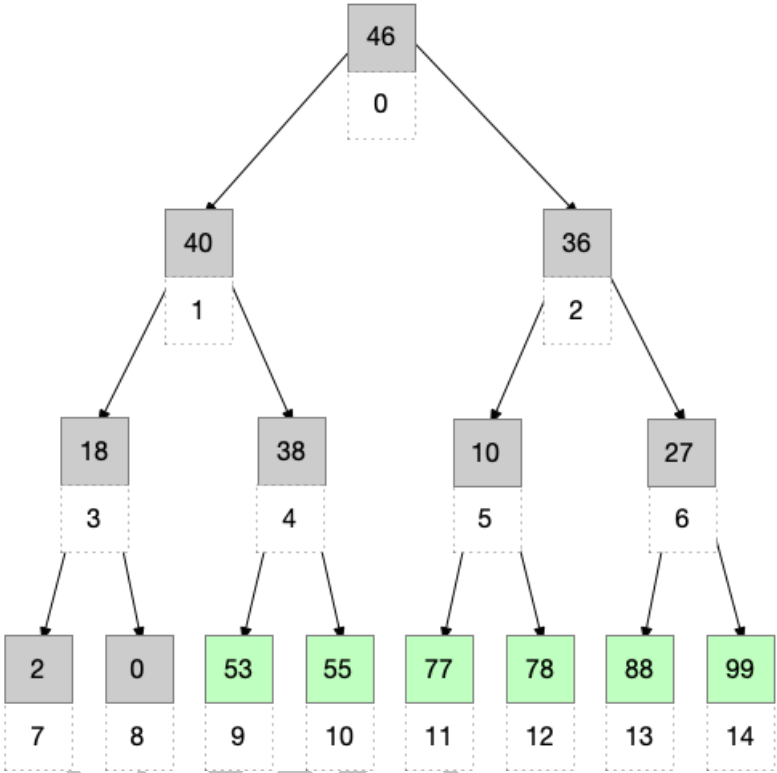
第五轮排序:

53	46	36	18	40	10	27	2	0	38	55	77	78	88	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



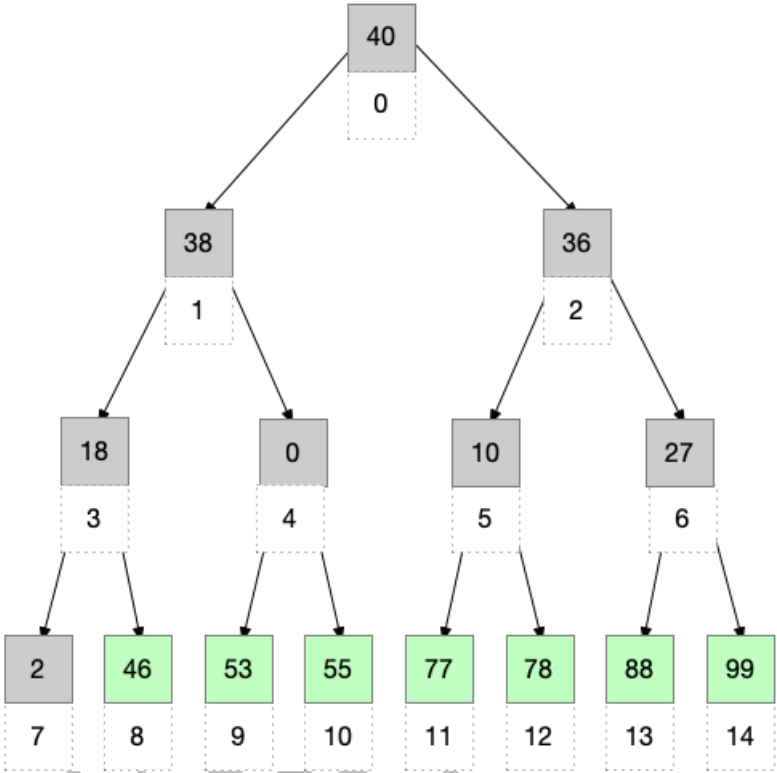
第六轮排序:

46	40	36	18	38	10	27	2	0	53	55	77	78	88	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



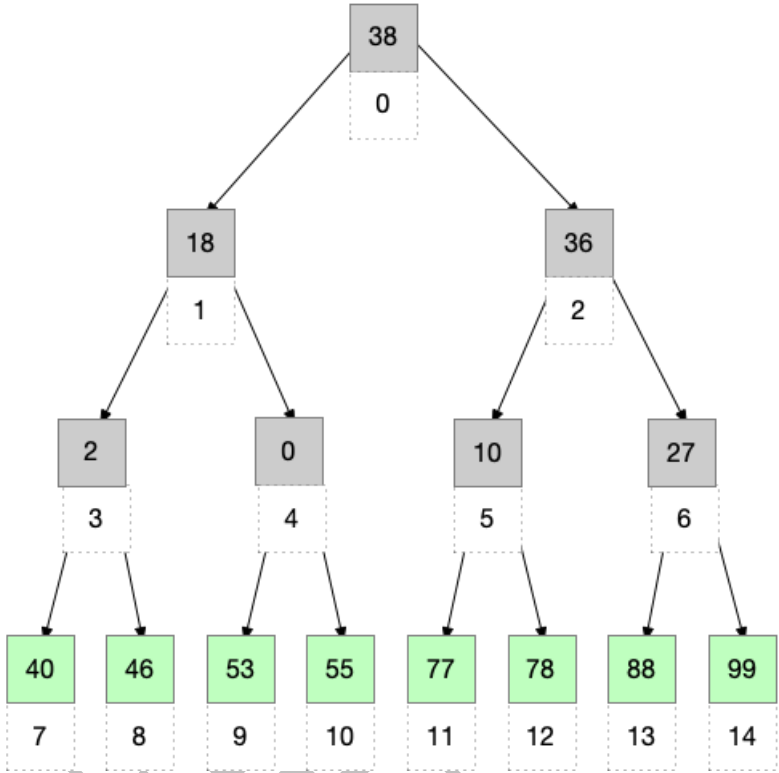
第七轮排序:

40	38	36	18	0	10	27	2	46	53	55	77	78	88	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



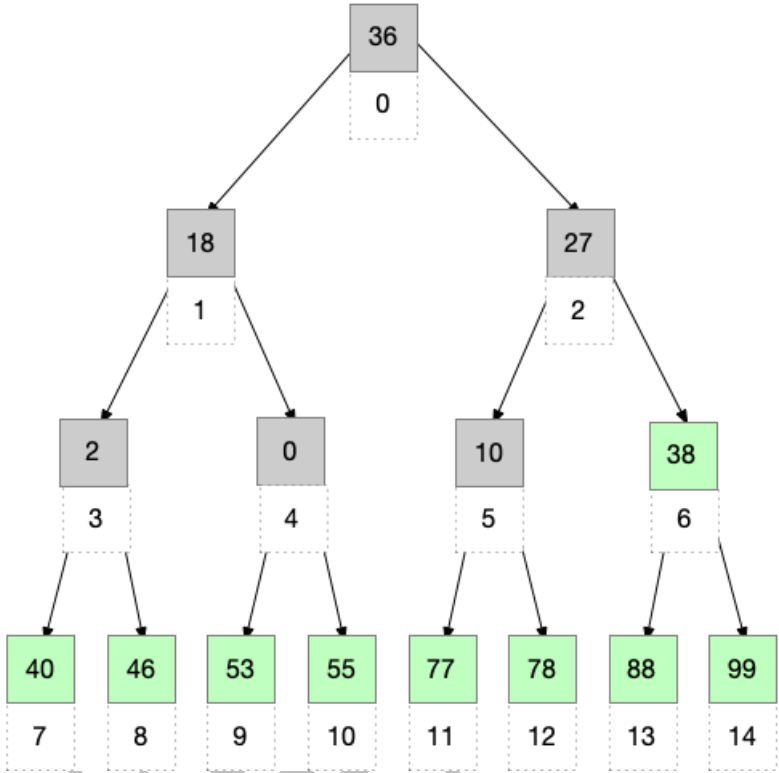
第八轮排序:

38	18	36	2	0	10	27	40	46	53	55	77	78	88	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



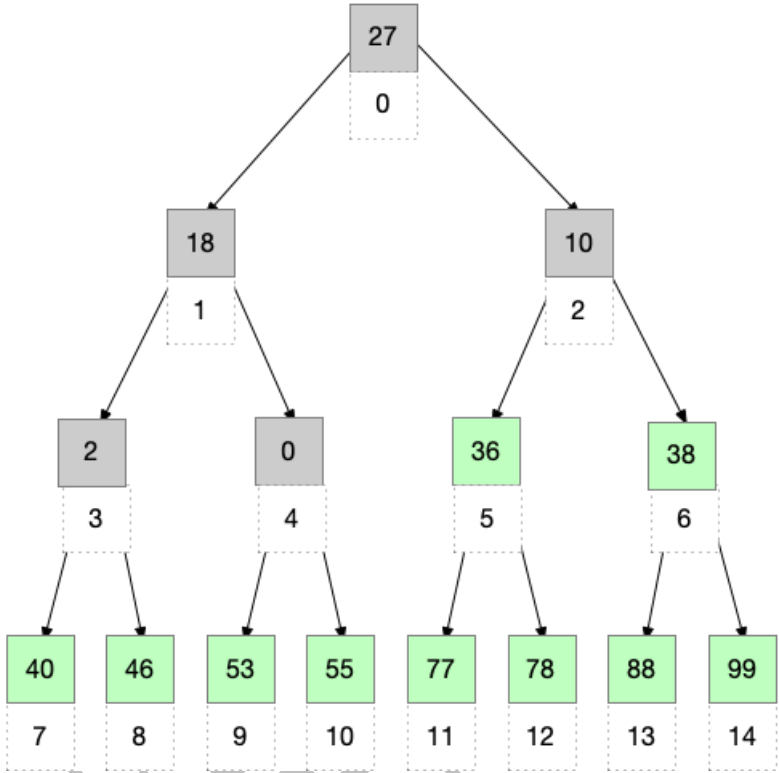
第九轮排序:

36	18	27	2	0	10	38	40	46	53	55	77	78	88	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



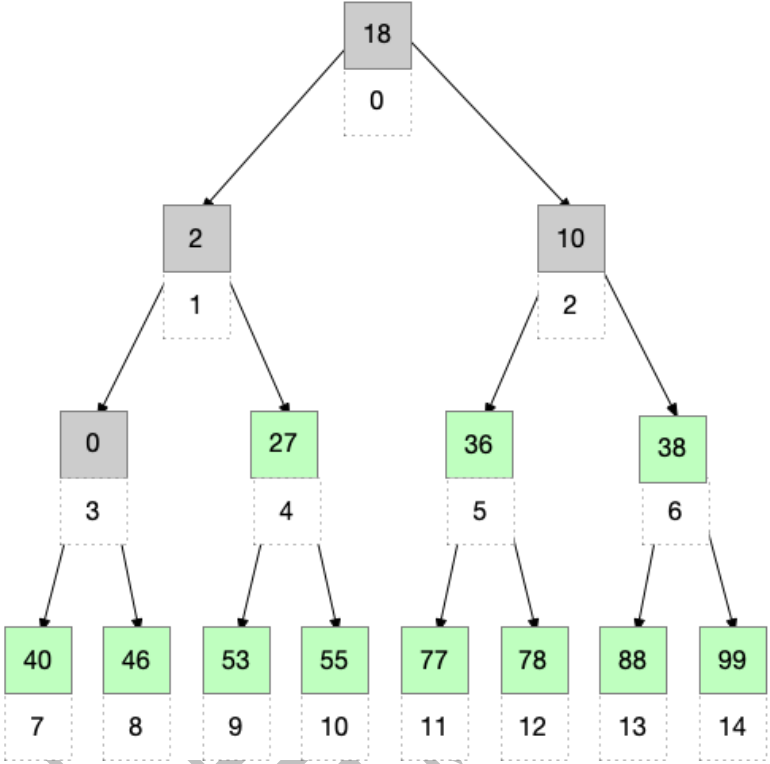
第十轮排序:

27	18	10	2	0	36	38	40	46	53	55	77	78	88	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



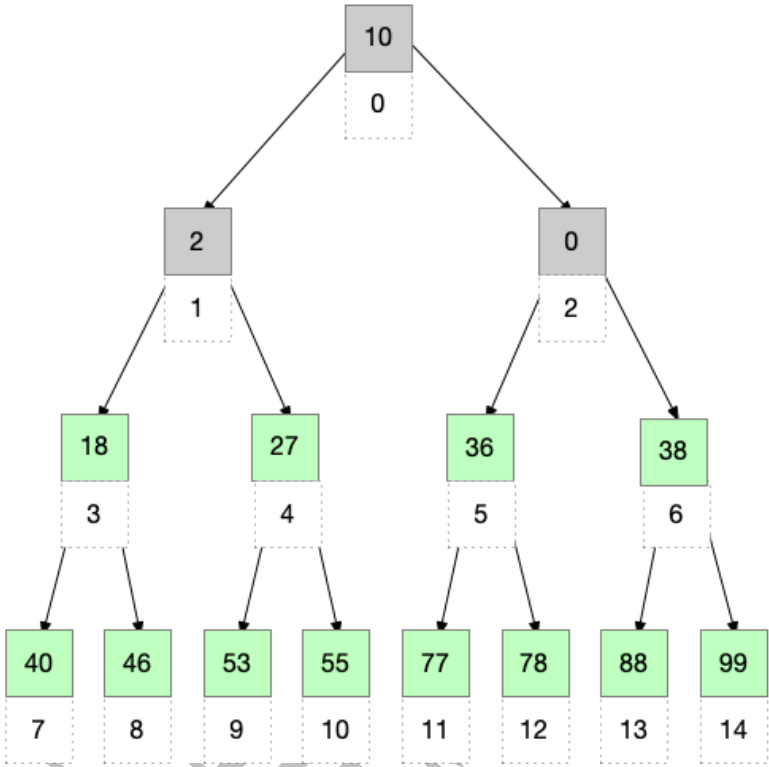
第十一轮排序:

18	2	10	0	27	36	38	40	46	53	55	77	78	88	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



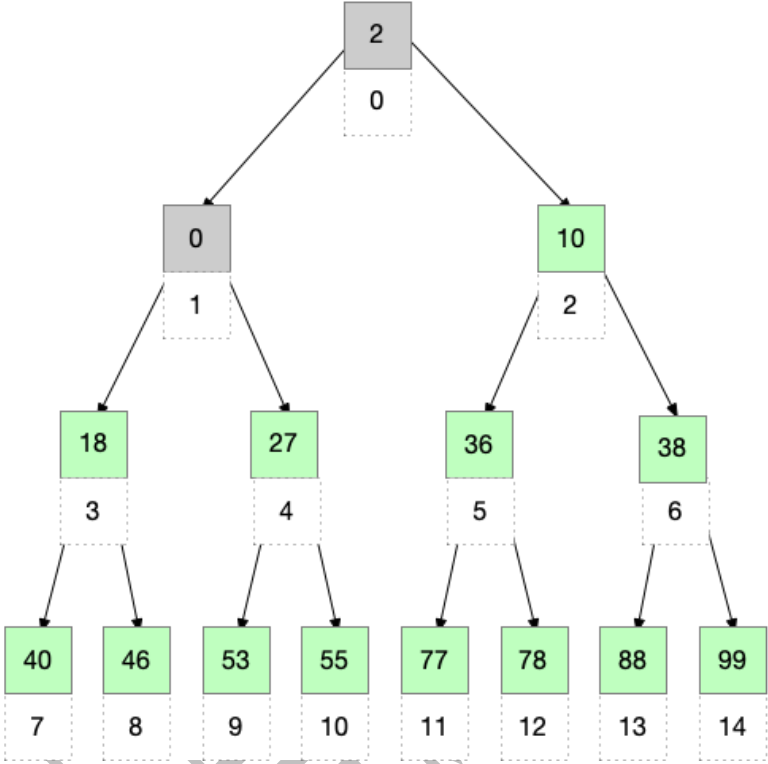
第十二轮排序:

10	2	0	18	27	36	38	40	46	53	55	77	78	88	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



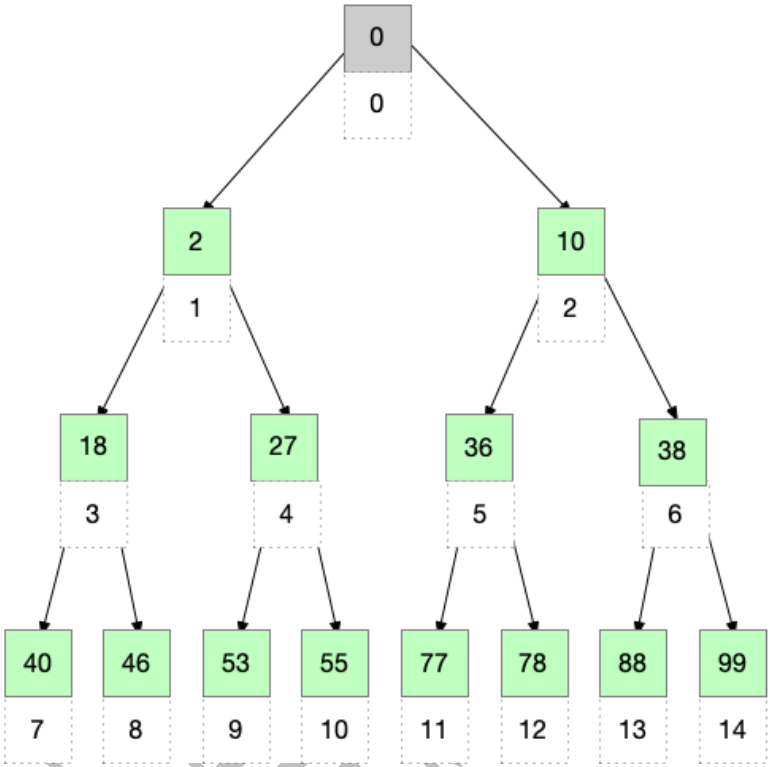
第十三轮排序：

2	0	10	18	27	36	38	40	46	53	55	77	78	88	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



第十四轮排序：

0	2	10	18	27	36	38	40	46	53	55	77	78	88	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



6.3 快速排序

6.3.1 自己设计一个长度不小于 10 的乱序数组，用快速排序，最终要生成升序数组

6.3.2 画出每一轮快速排序的状态

快速排序示例📌

初始状态 总长度=15	36	18	10	2	88	53	27	99	0	38	46	77	78	55	40
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
第一轮排序 红色为本轮枢轴元素	0	18	10	2	27	36	53	99	88	38	46	77	78	55	40
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
第二轮排序 红色为本轮枢轴元素 蓝色为之前几轮已确定最终位置的元素	0	18	10	2	27	36	40	46	38	53	88	77	78	55	99
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
第三轮排序 红色为本轮枢轴元素 蓝色为之前几轮已确定最终位置的元素	0	2	10	18	27	36	38	40	46	53	55	77	78	88	99
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
第四轮排序 红色为本轮枢轴元素 蓝色为之前几轮已确定最终位置的元素	0	2	10	18	27	36	38	40	46	53	55	77	78	88	99
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
第五轮排序 红色为本轮枢轴元素 蓝色为之前几轮已确定最终位置的元素	0	2	10	18	27	36	38	40	46	53	55	77	78	88	99
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
第六轮排序 红色为本轮枢轴元素 蓝色为之前几轮已确定最终位置的元素	0	2	10	18	27	36	38	40	46	53	55	77	78	88	99
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
快速排序结果	0	2	10	18	27	36	38	40	46	53	55	77	78	88	99
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

6.4 基数排序

6.4.1 自己设计一个长度不小于 15 的乱序链表，每个数据元素取值范围 0~99，用基数排序，最终要生成升序链表

6.4.2 画出每一轮基数排序的状态

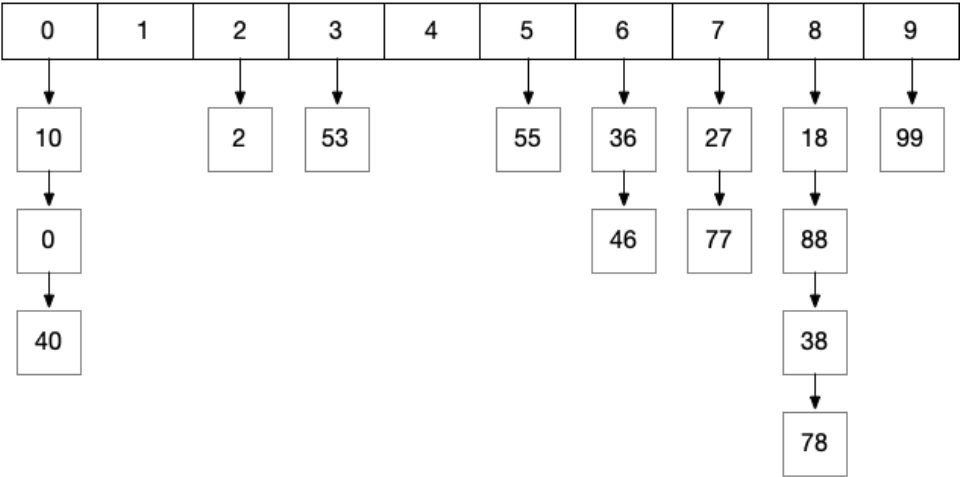
王道数据结构

基数排序示例📌

初始状态
总长度=15

36	18	10	2	88	53	27	99	0	38	46	77	78	55	40
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

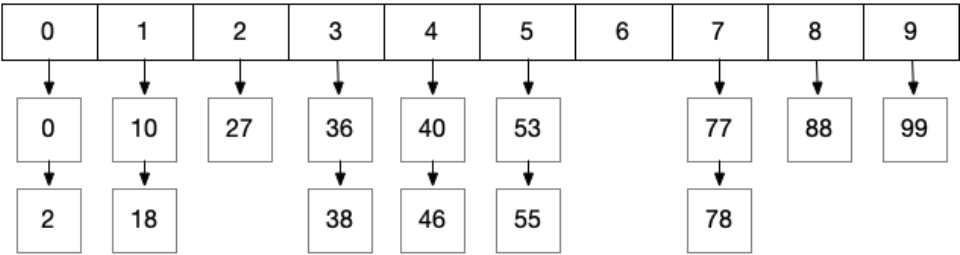
第一趟分配
(按个位分配)



第一趟收集
(按个位递增收集)

10	0	40	2	53	55	36	46	27	77	18	88	38	78	99
----	---	----	---	----	----	----	----	----	----	----	----	----	----	----

第二趟分配
(按十位分配)



第二趟收集
(按十位递增收集)

0	2	10	18	27	36	38	40	46	53	55	77	78	88	99
---	---	----	----	----	----	----	----	----	----	----	----	----	----	----